

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



**CyberCAN Defense: Phát hiện và phòng
thủ xâm nhập mạng nội bộ ô tô**

Nhóm 4

Phạm Thế Duy	23021787
Bùi Đăng Dương	23020795
Nguyễn Thị Ngọc Bích	23021770
Nguyễn Hữu Thái Dương	23021789

Cơ sở đo lường và điều khiển số

Lớp: ELT3207 1

Giảng viên: TS. Hoàng Gia Hưng

Mentor: Nguyễn Quang Khôi

Hà Nội, Tháng 6 năm 2026

Lời cam đoan

Chúng em cam đoan báo cáo “Hệ thống giám sát, phát hiện và phòng thủ xâm nhập CAN bus trên nền tảng ESP32” là kết quả học tập và nghiên cứu do chính nhóm thực hiện dưới sự hướng dẫn của giảng viên phụ trách học phần và các cá nhân hỗ trợ chuyên môn nếu có. Nội dung trình bày trong báo cáo được xây dựng trên cơ sở khảo sát tài liệu, phân tích mã nguồn, triển khai mô hình thực nghiệm và tổng hợp kết quả theo phạm vi mà nhóm đã thực hiện.

Nhóm cũng cam đoan các thông tin, hình ảnh, nhận xét và kết quả được nêu trong báo cáo là trung thực, phản ánh đúng quá trình thực hiện tại thời điểm biên soạn. Những nội dung tham khảo từ tài liệu, bài báo, báo cáo hoặc nguồn kỹ thuật khác đều cần được trích dẫn và ghi nhận phù hợp trong phần tài liệu tham khảo của báo cáo.

Nếu có sai sót liên quan đến tính trung thực học thuật, nguồn gốc nội dung hoặc việc sử dụng tài liệu tham khảo, chúng em xin chịu trách nhiệm trước giảng viên và quy định của học phần.

Lời cảm ơn

Trong quá trình thực hiện báo cáo và phát triển mô hình thực nghiệm về an ninh mạng CAN, nhóm đã nhận được nhiều sự hướng dẫn và hỗ trợ quý báu từ giảng viên, bạn học và những người đồng hành trong quá trình học tập. Chúng em xin bày tỏ sự biết ơn chân thành tới giảng viên phụ trách học phần vì đã định hướng đề tài, góp ý về nội dung chuyên môn và hỗ trợ nhóm trong quá trình hoàn thiện báo cáo.

Nhóm cũng xin cảm ơn các cá nhân và tập thể đã hỗ trợ về tài liệu kỹ thuật, thiết bị thực nghiệm, môi trường phát triển và những trao đổi chuyên môn liên quan đến hệ thống CAN, ESP32 cũng như các khía cạnh an ninh mạng nội bộ ô tô. Những góp ý đó đã giúp nhóm từng bước hoàn thiện mô hình, cải thiện cách trình bày và hiểu rõ hơn ý nghĩa thực tiễn của đề tài.

Do giới hạn về thời gian, phạm vi học phần và điều kiện thử nghiệm, báo cáo khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong tiếp tục nhận được các ý kiến nhận xét và góp ý để hoàn thiện hơn nội dung nghiên cứu cũng như rút kinh nghiệm cho các công việc học thuật và kỹ thuật sau này.

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Danh sách hình ảnh	vii
Danh sách liên kết	viii
Lời mở đầu	1
1 Giới thiệu chung	2
1.1 Bối cảnh	2
1.2 Giải pháp	3
2 Giao thức CAN	4
2.1 Khái niệm	4
2.1.1 Sơ đồ cấu trúc mạng CAN	4
2.1.2 Đặc điểm nổi bật của giao thức CAN	5
2.2 Nguyên tắc hoạt động	6
2.2.1 Cấu trúc khung dữ liệu (Data Frame)	6
2.2.2 Phân loại ID	6
2.2.3 CAN FD (Flexible Data-rate)	7
2.2.4 Truyền và nhận dữ liệu	8
2.2.5 Phát hiện lỗi	8
2.2.6 Giao tiếp đa điểm	8
2.3 Ứng dụng trong Automotive: Hệ thống ô tô	8

2.3.1	Các hệ thống tiêu biểu sử dụng giao thức CAN	9
2.3.2	Phân tầng mạng CAN trong ô tô	9
3	Bảo mật trong hệ thống mạng CAN	10
3.1	Các vấn đề về bảo mật trong hệ thống mạng CAN	10
3.2	Các phương pháp bảo mật trong hệ thống mạng CAN	11
3.2.1	Mã hóa thông tin (Encryption)	11
3.2.2	Xác thực thông điệp (Message Authentication)	11
3.2.3	Sử dụng bộ đếm tin nhắn đơn giản (Simple Message Counter)	12
3.2.4	Phát hiện xâm nhập (Intrusion Detection System - IDS)	12
3.2.5	Phân vùng mạng (Segmentation)	13
3.2.6	Chữ ký số (Digital Signatures)	13
3.2.7	Giao thức bảo mật lớp cao (Higher-layer Security Protocols)	14
4	CAN trong ESP32	15
4.1	Tổng quan về giao tiếp CAN trên ESP32	15
4.1.1	Bộ điều khiển TWAI	15
4.2	Kiến trúc một nút CAN dùng ESP32	16
4.2.1	Vai trò của bộ điều khiển CAN	16
4.2.2	Vai trò của bộ thu phát CAN	16
4.3	Kết nối phần cứng của mô hình	16
4.3.1	Ảnh xạ chân tín hiệu	17
4.4	Cấu trúc và nguyên lý hoạt động của TWAI	17
4.4.1	Các khối chức năng chính	17
4.4.2	Quy trình khởi tạo và trao đổi dữ liệu	18
4.5	Các tham số cấu hình quan trọng	18
4.5.1	Chế độ hoạt động	18
4.5.2	Tốc độ bit và sự đồng bộ trên bus	19
4.5.3	Bộ lọc khung và phạm vi tiếp nhận	19
4.6	Truyền nhận và xử lý lỗi trên ESP32	20
4.6.1	Quá trình truyền và nhận khung	20
4.6.2	Quản lý lỗi và trạng thái bus	20

4.9 Kết luận	21
5 Thiết kế hệ thống	22
5.1 Hệ thống phần cứng	22
5.1.1 ESP32	22
5.1.2 Module CAN TJA1050	23
5.1.3 Sơ đồ kết nối phần cứng	24
5.2 Hệ thống phần mềm	24
5.2.1 Firmware ESP32	25
5.2.2 Giao diện CAN và cơ chế bảo vệ bộ đếm	26
5.2.3 Hệ thống phát hiện xâm nhập (IDS)	26
5.2.4 Bộ mô phỏng tấn công	27
5.2.5 Ứng dụng web giám sát và điều khiển	27
5.2.6 Quản lý trạng thái và lưu trữ dữ liệu	28
5.3 Nguyên lý hoạt động của hệ thống	29
5.3.1 Giao tiếp CAN cơ bản	29
5.3.2 Cơ chế bảo vệ bộ đếm thông điệp	29
5.3.3 Replay Attack	29
5.3.4 Spoofing Attack	30
5.3.5 DoS Attack	30
5.3.6 Fuzzing Attack	31
5.3.7 Cơ chế phòng thủ chủ động	31
6 Thực thi, đo lường và đánh giá	33
I. Giao tiếp 2 chiều	33
II. Các kịch bản tấn công và phản ứng phòng thủ	34
1. Tấn công DoS	35
2. Tấn công Replay	37
3. Tấn công Fuzzing	39
4. Tấn công Spoofing	41
III. Video minh họa các kịch bản	43

7	Tổng kết và hướng phát triển	44
7.1	Tổng kết	44
7.2	Hướng phát triển trong tương lai	45
	Tài liệu tham khảo	46

Danh sách hình ảnh

2.1	Mạng CAN tổng quát	5
2.2	Standard CAN: 11-Bit Identifier	7
2.3	Extended CAN: 29-Bit Identifier	7
2.4	Flexible Data CAN	8
4.1	Sơ đồ khối của một nút CAN dùng ESP32	16
4.2	Quy trình tổng quát khi khởi tạo và vận hành TWAI trên ESP32	18
5.1	Module CAN TJA1050	23
5.2	Sơ đồ khối phần cứng tổng quan	24
5.3	Kiến trúc tổng thể hai lớp phần mềm của hệ thống	25
5.4	Giao diện web giám sát, tấn công và phòng thủ CAN	28
5.5	Sơ đồ luồng xử lý tổng thể của hệ thống	29
6.1	Mô hình phần cứng giao tiếp CAN với 2 node ESP32	33
6.2	Phiên giao tiếp CAN hai chiều bình thường hiển thị trên giao diện web	34
6.3	Mô hình phần cứng thực tế khi bổ sung thêm node hacker vào bus CAN	35
6.4	Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công DoS	36
6.5	Giao diện sau khi bật defense để chặn lưu lượng DoS	37
6.6	Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Replay	38
6.7	Giao diện sau khi bật defense để chặn khung replay	39
6.8	Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Fuzzing	40
6.9	Giao diện sau khi bật defense để chặn lưu lượng Fuzzing	41
6.10	Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Spoofing	42
6.11	Giao diện sau khi bật defense để chặn khung spoofing	43

Danh sách liên kết

- [Github](#)
- [Thư mục video minh họa các kịch bản tấn công và phòng thủ DoS, Replay, Fuzzing, Spoofing](#)

Lời mở đầu

Mạng truyền thông *Controller Area Network* (CAN) là một thành phần quan trọng trong các hệ thống ô tô và những thời gian thực nhờ khả năng truyền dữ liệu tin cậy, độ trễ thấp và cơ chế ưu tiên bản tin theo mã định danh. Tuy nhiên, CAN ban đầu không được thiết kế với trọng tâm là an toàn thông tin, nên vẫn tồn tại nhiều hạn chế như thiếu xác thực nguồn gửi, không mã hóa dữ liệu và khó ngăn chặn các hành vi phát lại, giả mạo hoặc gây ngập lưu lượng trên bus. Khi số lượng thiết bị kết nối ngày càng tăng, các rủi ro này trở thành một vấn đề cần được quan tâm trong quá trình nghiên cứu và triển khai hệ thống.

Từ thực tế đó, đề tài xây dựng mô hình thử nghiệm *CyberCAN Defense* trên nền tảng ESP32 kết hợp transceiver CAN nhằm mô phỏng quá trình truyền nhận bản tin, các kịch bản tấn công cơ bản và cơ chế phát hiện bất thường trên bus. Hệ thống được phát triển theo hướng gọn nhẹ nhưng vẫn hỗ trợ các chức năng chính như giám sát lưu lượng CAN, mô phỏng replay, spoofing và flood, đồng thời cung cấp giao diện web để theo dõi cảnh báo và điều khiển tập trung. Báo cáo tập trung trình bày cơ sở lý thuyết của giao thức CAN, các vấn đề an ninh liên quan, cấu trúc phần cứng dùng ESP32 và nguyên lý hoạt động của hệ thống đã xây dựng.

Chapter 1

Giới thiệu chung

1.1 Bối cảnh

Trong kỷ nguyên của xe hơi hiện đại, các hệ thống điều khiển điện tử (ECU - Electronic Control Unit) đóng vai trò trung tâm trong việc điều phối nhiều chức năng khác nhau, từ điều hòa không khí, đèn pha cho đến các hệ thống phức tạp như hỗ trợ lái tự động (ADAS), chống bó cứng phanh (ABS), kiểm soát lực kéo (TCS) và túi khí (Airbag). Để các ECU này có thể giao tiếp hiệu quả, giao thức *Controller Area Network* (CAN) được sử dụng như xương sống truyền thông nội bộ trong ô tô. CAN cung cấp cơ chế truyền thông đa điểm tốc độ cao, đáng tin cậy và có tính thời gian thực giữa các bộ điều khiển.

Tuy nhiên, khi các phương tiện ngày càng trở nên kết nối nhiều hơn thông qua cổng OBD-II, Bluetooth, Wi-Fi, 4G/5G và cả cập nhật OTA, nguy cơ bị tấn công mạng cũng tăng lên đáng kể. Mặc dù CAN được thiết kế tối ưu cho độ tin cậy và tính thời gian thực, giao thức này không tích hợp sẵn các cơ chế bảo mật cơ bản như:

- Không có cơ chế xác thực giữa các node.
- Không có mã hóa dữ liệu truyền trên bus.
- Không có phân quyền truy cập giữa các thành phần tham gia mạng.

Do đó, kẻ tấn công chỉ cần có khả năng truy cập vật lý hoặc gián tiếp vào mạng CAN là đã có thể thực hiện các hành vi nguy hiểm như:

- Giả mạo gói tin (*spoofing*) để điều khiển ECU trái phép.
- Tấn công từ chối dịch vụ (*DoS*) làm treo hoặc quá tải hệ thống.
- Nghe lén dữ liệu (*eavesdropping*) để phân tích hành vi của xe.

Trước bối cảnh đó, yêu cầu cấp thiết đặt ra là cần phát triển các giải pháp bảo mật cho mạng CAN. Trên thực tế, nhiều hướng tiếp cận đã được nghiên cứu và áp dụng, chẳng hạn như hệ thống phát hiện xâm nhập (IDS), mã hóa giao tiếp trên CAN hoặc phân tích hành vi bất thường trong quá trình vận hành.

1.2 Giải pháp

Trước các mối đe dọa an ninh mạng ngày càng gia tăng đối với hệ thống xe hơi, đặc biệt là với CAN bus, nhiều giải pháp bảo mật đã được nghiên cứu và triển khai trên toàn thế giới. Các giải pháp này tập trung vào việc phát hiện, ngăn chặn và giảm thiểu tác động của các cuộc tấn công vào hệ thống điện tử trên xe.

Dưới đây là các nhóm giải pháp chính:

- a. **Hệ thống phát hiện xâm nhập (IDS - Intrusion Detection System):** IDS là một trong những hướng tiếp cận phổ biến nhất trong bảo mật mạng CAN. Hệ thống này giám sát dữ liệu truyền trên bus và tìm kiếm các dấu hiệu bất thường dựa trên phân tích thống kê tần suất gói tin, phân tích hành vi hoặc các kỹ thuật học máy để phân loại hoạt động hợp lệ và độc hại.
- b. **Mã hóa dữ liệu truyền trên CAN bus:** Một số nghiên cứu đề xuất áp dụng kỹ thuật mã hóa nhẹ (*lightweight encryption*) để bảo vệ nội dung gói tin truyền qua mạng CAN. Tuy nhiên, giải pháp này gặp nhiều thách thức do giới hạn băng thông của CAN và yêu cầu thời gian thực của hệ thống xe.
- c. **Xác thực thông điệp (Message Authentication):** Thay vì chỉ mã hóa, một số giải pháp tập trung vào việc gắn mã xác thực hoặc chữ ký số nhẹ vào các gói tin CAN nhằm đảm bảo dữ liệu đến từ nguồn hợp lệ và chưa bị chỉnh sửa. Điều này đặc biệt hữu ích để chống lại các cuộc tấn công *spoofing* và *replay*.
- d. **Giám sát hành vi dựa trên định thời (Timing-based Anomaly Detection):** Nhiều cuộc tấn công làm thay đổi chu kỳ truyền hoặc độ trễ của gói tin. Vì vậy, việc theo dõi thời gian truyền và kiểm tra sự bất thường trong mẫu định thời là một kỹ thuật hữu ích để phát hiện xâm nhập.

Mặc dù nhiều giải pháp bảo mật mạng CAN đã được đề xuất và triển khai, không có giải pháp nào là hoàn hảo trong mọi bối cảnh. Hướng đi khả thi nhất hiện nay là kết hợp nhiều lớp bảo mật, từ mã hóa và xác thực đến phát hiện xâm nhập, đồng thời thiết kế hệ thống mạng thông minh và có phân vùng để nâng cao mức độ an toàn cho xe hơi hiện đại.

Chapter 2

Giao thức CAN

2.1 Khái niệm

CAN (Controller Area Network) là một giao thức truyền thông nối tiếp được phát triển bởi hãng Bosch vào những năm 1980 để giải quyết nhu cầu trao đổi dữ liệu giữa các thiết bị điện tử trong xe ô tô. Trước khi có CAN, các hệ thống điều khiển trong xe phải kết nối trực tiếp với nhau bằng dây dẫn, gây tốn kém và khó bảo trì. Việc sử dụng CAN cho phép các bộ điều khiển (ECU) chia sẻ dữ liệu với nhau thông qua một đường truyền chung, từ đó giảm thiểu số lượng dây, tăng độ tin cậy và khả năng mở rộng hệ thống.

Giao thức CAN hiện được chuẩn hóa trong tiêu chuẩn ISO 11898 và đã vượt ra khỏi ngành công nghiệp ô tô, được ứng dụng rộng rãi trong công nghiệp tự động hoá, robot, y tế, hàng không và các hệ thống nhúng.

2.1.1 Sơ đồ cấu trúc mạng CAN

Dưới đây là sơ đồ kết nối tổng quát giữa các Node thông qua bộ điều khiển (CAN Controller) và bộ thu phát (CAN Transceiver) trên đường truyền vật lý:

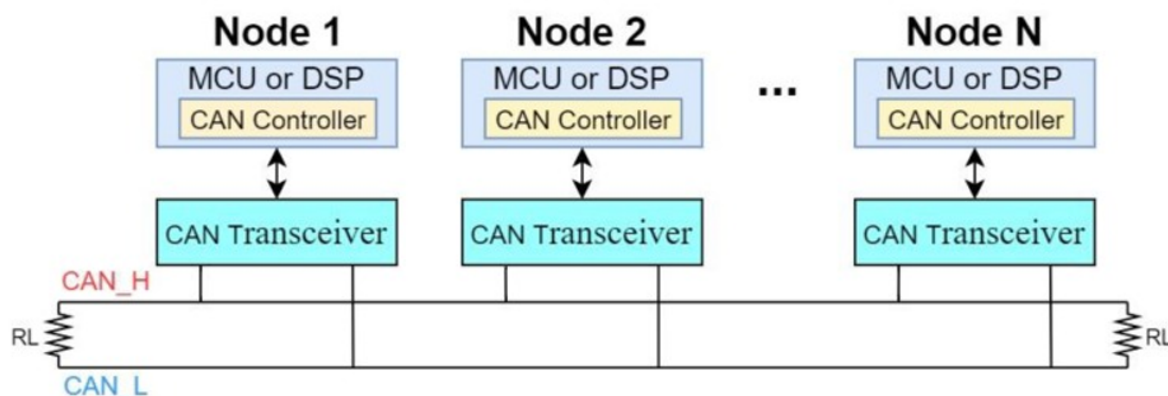


Figure 2.1: Mạng CAN tổng quát

2.1.2 Đặc điểm nổi bật của giao thức CAN

- **Hoạt động theo mô hình đa master (multi-master):** Mọi thiết bị (node) trên mạng đều có quyền khởi tạo truyền dữ liệu. Điều này giúp mạng CAN linh hoạt hơn so với các mô hình truyền thông chỉ có một master điều khiển.
- **Truyền thông không định địa chỉ thiết bị:** Khác với giao thức UART hoặc I2C, CAN không dùng địa chỉ thiết bị. Thay vào đó, mỗi khung dữ liệu mang một ID (identifier) đại diện cho nội dung thông điệp. Các node sẽ lọc và xử lý dữ liệu theo ID mà chúng quan tâm. Điều này giúp tăng tính linh hoạt và khả năng mở rộng hệ thống.
- **Cơ chế ưu tiên truyền:** Trong trường hợp nhiều node cùng muốn truyền dữ liệu, giao thức CAN sẽ so sánh ID theo bit từng bước để quyết định thiết bị nào được phép tiếp tục truyền. ID có giá trị nhị phân thấp hơn (ưu tiên cao hơn) sẽ giành quyền truy cập bus. Quá trình này gọi là arbitration và diễn ra hoàn toàn tự động mà không gây xung đột.
- **Phát hiện và xử lý lỗi mạnh mẽ:** CAN có nhiều cơ chế phát hiện lỗi như:
 - *Bit monitoring:* Kiểm tra sự khác biệt giữa bit gửi và nhận.
 - *CRC check:* Kiểm tra tính toàn vẹn khung dữ liệu.
 - *ACK check:* Đảm bảo dữ liệu được ít nhất một node khác nhận thành công.
 - *Form check:* Kiểm tra định dạng của khung dữ liệu.

Nếu phát hiện lỗi, node sẽ tự động phát một Error Frame và yêu cầu gửi lại.

- **Tốc độ truyền dữ liệu linh hoạt:** Giao thức CAN hỗ trợ nhiều tốc độ khác nhau tùy theo chiều dài đường truyền. Trên thực tế:
 - Với khoảng cách ngắn (dưới 40m), có thể đạt tốc độ tối đa 1 Mbps.
 - Với khoảng cách dài (lên đến 1 km), tốc độ có thể giảm xuống khoảng 50 kbps.

Điều này làm cho CAN phù hợp với cả hệ thống nhỏ lẫn hệ thống phân tán rộng.

- **Giao tiếp đồng bộ và thời gian thực:** CAN sử dụng chuẩn mã hóa NRZ (Non-Return to Zero) và kỹ thuật bit stuffing để đồng bộ hóa giữa các node mà không cần xung clock ngoài. Điều này giúp đảm bảo giao tiếp thời gian thực, đặc biệt quan trọng trong các hệ thống an toàn như phanh ABS, điều khiển động cơ hay túi khí.
- **Chi phí thấp, độ tin cậy cao:** Nhờ vào khả năng xử lý lỗi, giảm thiểu dây nối và chi phí phần cứng, CAN trở thành lựa chọn hàng đầu cho các hệ thống nhúng cần độ tin cậy cao với chi phí hợp lý.

2.2 Nguyên tắc hoạt động

2.2.1 Cấu trúc khung dữ liệu (Data Frame)

Trường	Vai trò
Start of Frame	Xác định bắt đầu một khung dữ liệu.
Arbitration Field	Chứa ID và bit RTR (Remote Transmission Request).
Control Field	Xác định độ dài dữ liệu.
Data Field	Dữ liệu truyền đi (từ 0 đến 8 byte).
CRC Field	Dùng để kiểm tra lỗi dữ liệu truyền.
ACK Field	Thiết bị nhận phản hồi xác nhận khung hợp lệ.
End of Frame	Đánh dấu kết thúc khung dữ liệu.

Table 2.1: Cấu trúc chi tiết các trường trong Data Frame

2.2.2 Phân loại ID

Trong giao thức CAN, mỗi khung dữ liệu mang một mã định danh (ID - Identifier) thay vì địa chỉ thiết bị. ID được sử dụng để xác định nội dung thông điệp, phân mức ưu tiên trong quá trình tranh chấp truyền (arbitration) và giúp các node lọc dữ liệu cần thiết.

- **Standard ID (11 bit):** Là định dạng gốc của chuẩn CAN theo ISO 11898-1, còn gọi là Base Frame Format. Định dạng này gồm 11 bit, cho phép tối đa $2^{11} = 2048$ mã định danh khác nhau. Thường được sử dụng rộng rãi trong các hệ thống nhỏ, số lượng thiết bị hoặc thông điệp ít. Khung dữ liệu ngắn giúp giảm thời gian truyền, tăng hiệu quả mạng.
quát

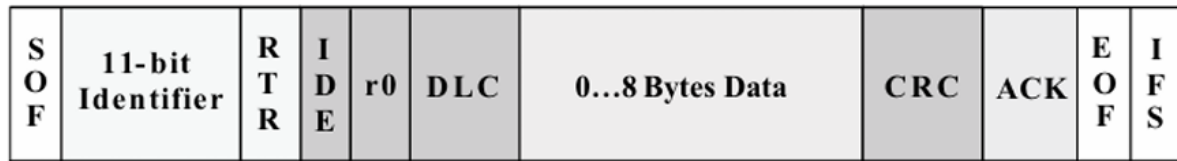


Figure 2.2: Standard CAN: 11-Bit Identifier

- **Extended ID (29 bit):** Mở rộng từ chuẩn CAN cơ bản để đáp ứng nhu cầu phức tạp hơn. Cấu trúc gồm 11 bit gốc + 18 bit mở rộng, tổng cộng 29 bit. Định dạng này cho phép tối đa $2^{29} \approx 537$ triệu mã định danh khác nhau. Thường dùng trong các hệ thống lớn như xe hiện đại, hệ thống công nghiệp quy mô lớn. Hệ thống phân biệt bằng bit điều khiển IDE (Identifier Extension): nếu $IDE = 1 \Rightarrow$ sử dụng Extended ID.

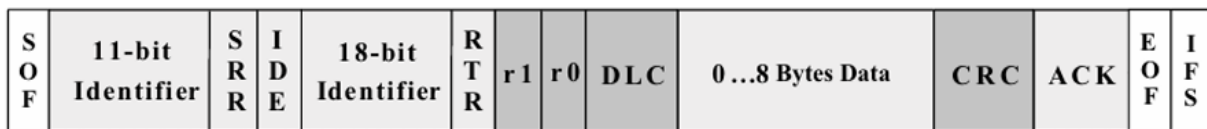


Figure 2.3: Extended CAN: 29-Bit Identifier

2.2.3 CAN FD (Flexible Data-rate)

CAN FD là phiên bản mở rộng của chuẩn CAN cổ điển, do Bosch phát triển vào năm 2012 và được tiêu chuẩn hóa trong ISO 11898-1:2015. Nó khắc phục các hạn chế về tốc độ và dung lượng dữ liệu với những điểm cải tiến chính:

- **Tăng dung lượng dữ liệu:** CAN cổ điển giới hạn ở 8 byte dữ liệu. CAN FD hỗ trợ tối đa 64 byte dữ liệu mỗi khung.
- **Tốc độ truyền linh hoạt:** Giai đoạn tranh chấp (arbitration phase) giữ tốc độ thấp để đảm bảo tương thích và đồng bộ. Giai đoạn truyền dữ liệu (data transmission phase) có thể đạt tốc độ cao hơn rất nhiều (tối đa 8 Mbps hoặc hơn).
- **Tính tương thích ngược:** Có thể hoạt động chung với thiết bị chỉ hỗ trợ CAN cổ điển nhờ bit FDF (FD Format) giúp thiết bị cổ điển nhận biết và bỏ qua khung FD.
- **Cấu trúc khung mới:** Bổ sung trường FDF, BRS (Bit Rate Switch) để xác định khung FD và tốc độ cao. Đồng thời mã kiểm tra lỗi CRC tăng lên 17-bit hoặc 21-bit tùy theo chiều dài thực tế của dữ liệu.

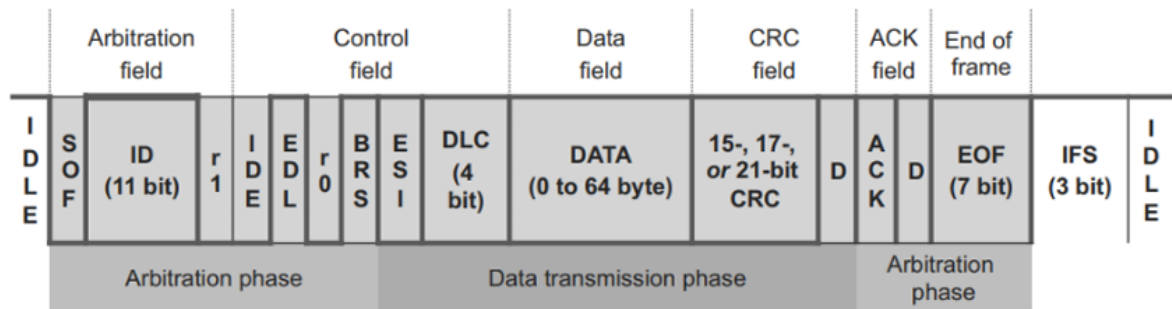


Figure 2.4: Flexible Data CAN

2.2.4 Truyền và nhận dữ liệu

- Tất cả thiết bị đều giám sát liên tục bus CAN.
- Thiết bị bắt đầu truyền khi phát hiện bus rảnh.
- Cơ chế arbitration dựa vào ID: thiết bị có ID nhỏ hơn sẽ được ưu tiên.
- Sử dụng mã hóa NRZ và kỹ thuật bit stuffing để đảm bảo đồng bộ.

2.2.5 Phát hiện lỗi

- **Bit Monitoring:** So sánh bit truyền và bit nhận trên bus.
- **CRC Checking:** Kiểm tra tính toàn vẹn của dữ liệu.
- **Form Checking:** Đảm bảo khung dữ liệu đúng định dạng.
- **ACK Checking:** Thiết bị nhận phải xác nhận bằng cách gửi ACK.

2.2.6 Giao tiếp đa điểm

Giao tiếp theo mô hình bus, nơi nhiều node kết nối chung trên hai dây tín hiệu CAN (CAN_H và CAN_L). Hai đầu của bus cần gắn điện trở 120 Ω để triệt tiêu phản xạ và đảm bảo tín hiệu ổn định.

2.3 Ứng dụng trong Automotive: Hệ thống ô tô

Trong các phương tiện giao thông hiện đại, đặc biệt là ô tô, giao thức CAN đóng vai trò là xương sống trong mạng truyền thông nội bộ. Với khả năng truyền thông tin hiệu quả, đáng

tin cậy và chi phí thấp, CAN đã trở thành tiêu chuẩn công nghiệp trong các hệ thống điện tử trên xe.

2.3.1 Các hệ thống tiêu biểu sử dụng giao thức CAN

- **ECU động cơ (Engine Control Unit):** Điều khiển hoạt động của động cơ như phun nhiên liệu, đánh lửa, kiểm soát khí xả, v.v.
- **Hệ thống túi khí (Airbag System):** Giao tiếp giữa các cảm biến va chạm, bộ xử lý và bộ kích hoạt túi khí để đảm bảo phản ứng tức thời khi xảy ra va chạm.
- **Hệ thống phanh ABS/ESP:** Trao đổi dữ liệu liên tục giữa cảm biến tốc độ bánh xe, ECU phanh và các mô-đun ổn định thân xe để ngăn trượt và lật xe.
- **Hệ thống điều khiển tiện nghi:** Bao gồm điều hòa không khí, điều chỉnh ghế, gương, cửa sổ, đèn nội thất, thường kết nối qua mạng CAN tốc độ thấp (Low-Speed CAN).
- **Màn hình thông tin và giải trí (Infotainment):** Kết nối giữa bảng điều khiển trung tâm, màn hình, bộ định vị GPS, hệ thống âm thanh, và các nút điều khiển trên vô lăng.

2.3.2 Phân tầng mạng CAN trong ô tô

Trong thực tế, các hệ thống điện tử trong xe thường được chia thành các tầng mạng CAN khác nhau, tùy vào tốc độ và mức độ quan trọng:

- **High-Speed CAN (500 kbps - 1 Mbps):** Dùng cho các hệ thống an toàn và điều khiển động cơ, yêu cầu tốc độ cao và phản hồi nhanh.
- **Low-Speed / Fault-Tolerant CAN (125 kbps):** Dùng cho các chức năng thân xe như đèn, gương, khóa cửa, nơi yêu cầu tốc độ không quá cao nhưng độ ổn định quan trọng.
- **CAN FD (Flexible Data-rate):** Được tích hợp trong các xe mới để truyền lượng dữ liệu lớn hơn (như từ camera, radar) với tốc độ cao hơn và độ trễ thấp hơn.

Chapter 3

Bảo mật trong hệ thống mạng CAN

3.1 Các vấn đề về bảo mật trong hệ thống mạng CAN

Mạng CAN (*Controller Area Network*) được thiết kế từ đầu nhằm tối ưu cho tốc độ truyền thông và độ tin cậy trong các hệ thống nhúng, đặc biệt là ô tô. Tuy nhiên, thiết kế ban đầu của CAN không tập trung vào các yêu cầu bảo mật, vì vậy giao thức này tồn tại nhiều lỗ hổng tiềm ẩn khi được đặt trong môi trường kết nối hiện đại.

Dưới đây là một số vấn đề bảo mật chính của mạng CAN:

- **Không có cơ chế xác thực (Authentication):** Bất kỳ thiết bị nào được kết nối vào bus CAN đều có thể gửi thông điệp. Hệ thống không có cách tự nhiên để xác minh nguồn gốc thực sự của thông điệp đó.
- **Không có mã hóa (Encryption):** Dữ liệu trên bus CAN được truyền ở dạng rõ ràng (*plain-text*), tạo điều kiện cho kẻ tấn công nghe lén và phân tích nội dung.
- **Không có phân quyền truy cập (Access Control):** Tất cả các node trên mạng đều chia sẻ cùng một môi trường truyền thông, không có cơ chế kiểm soát vai trò hay đặc quyền truy cập.
- **Dễ bị tấn công từ chối dịch vụ (Denial-of-Service - DoS):** Một thiết bị độc hại có thể liên tục gửi các frame có ID ưu tiên cao để chiếm quyền truy cập bus, làm gián đoạn hoạt động của các node khác.
- **Khả năng nghe trộm (Eavesdropping):** Vì toàn bộ bus chia sẻ chung tín hiệu, mọi node đều có thể quan sát các thông điệp đang lưu thông trên mạng.
- **Dễ bị giả mạo (Spoofing):** Một thiết bị độc hại có thể giả mạo ID của một node hợp lệ để phát thông điệp sai lệch lên mạng.

3.2 Các phương pháp bảo mật trong hệ thống mạng CAN

Để khắc phục các hạn chế nêu trên, nhiều giải pháp bảo mật đã được đề xuất. Mỗi phương pháp có ưu điểm và đánh đổi riêng, tùy thuộc vào khả năng phần cứng, yêu cầu thời gian thực và kiến trúc tổng thể của hệ thống.

3.2.1 Mã hóa thông tin (Encryption)

Nguyên lý. Dữ liệu truyền trên bus CAN được mã hóa để ngăn chặn việc bị nghe lén hoặc chỉnh sửa trái phép. Nhờ đó, chỉ các thiết bị hợp lệ mới có thể giải mã và sử dụng thông điệp.

Cách thực hiện.

- *Mã hóa đối xứng (AES):* Tất cả các node chia sẻ một khóa bí mật chung. Mỗi thông điệp được mã hóa trước khi truyền và chỉ các node có khóa đúng mới có thể giải mã.
- *Mã hóa bất đối xứng (RSA):* Mỗi node có một cặp khóa công khai và khóa bí mật. Node gửi dùng khóa công khai của node nhận để mã hóa, còn node nhận dùng khóa bí mật để giải mã.

Ví dụ. Một node gửi dữ liệu D được mã hóa thành $E_K(D)$ bằng AES-128. Node nhận dùng cùng khóa K để giải mã và thu lại dữ liệu gốc.

Ưu điểm. Đảm bảo tính riêng tư và an toàn của thông điệp. Giảm nguy cơ nghe lén và giả mạo nội dung.

Nhược điểm. Tốn tài nguyên xử lý và bộ nhớ, đặc biệt với các thuật toán nặng như RSA. Có thể làm tăng độ trễ truyền thông nếu không được tối ưu. Việc quản lý khóa trong môi trường nhúng tương đối phức tạp.

3.2.2 Xác thực thông điệp (Message Authentication)

Nguyên lý. Một mã xác thực thông điệp (MAC - *Message Authentication Code*) được gắn thêm vào dữ liệu để kiểm tra nguồn gốc và tính toàn vẹn của thông điệp.

Cách thực hiện.

- Sử dụng hàm băm kết hợp với khóa bí mật để tạo mã xác thực, ví dụ như HMAC-SHA256.
- Node gửi tính giá trị MAC từ dữ liệu và truyền kèm trong payload.
- Node nhận tính lại MAC và so sánh với giá trị đã nhận; nếu trùng khớp, thông điệp được xem là hợp lệ.

Ví dụ. Với dữ liệu D và khóa K , node gửi tính $MAC = HMAC_K(D)$ rồi gửi cặp (D, MAC) . Node nhận tính lại $HMAC_K(D)$ để kiểm tra.

Ưu điểm. Ngăn chặn giả mạo thông điệp. Phát hiện được các sửa đổi trái phép trong quá trình truyền.

Nhược điểm. Không tự chống được tấn công phát lại nếu không kết hợp thêm bộ đếm hoặc dấu thời gian. Làm tăng kích thước payload, dễ chạm giới hạn 8 byte của CAN 2.0. Tăng tải tính toán trên vi điều khiển.

3.2.3 Sử dụng bộ đếm tin nhắn đơn giản (Simple Message Counter)

Nguyên lý. Mỗi thông điệp được gán một bộ đếm tăng dần để đảm bảo tính “tươi mới” (*freshness*). Thiết bị nhận dùng giá trị này để phát hiện và từ chối các bản tin bị phát lại.

Cách thực hiện.

- Gắn một trường bộ đếm nhỏ, ví dụ 8 bit hoặc 16 bit, vào thông điệp M_k .
- Thiết bị nhận lưu giá trị bộ đếm gần nhất cho từng ID hoặc từng phiên giao tiếp.
- Nếu bộ đếm trong thông điệp mới không lớn hơn giá trị đã lưu, thông điệp có thể bị xem là bản sao hoặc lỗi thời.

Ví dụ. Nếu thông điệp trước đó có bộ đếm là 5 thì thông điệp kế tiếp hợp lệ phải có bộ đếm bằng 6 hoặc lớn hơn.

Ưu điểm. Cơ chế đơn giản, hiệu quả và phù hợp với vi điều khiển giới hạn tài nguyên. Giúp chống lại các cuộc tấn công phát lại ở mức cơ bản.

Nhược điểm. Không tự bảo vệ trước giả mạo hoặc nghe lén nếu không kết hợp với MAC hay mã hóa. Yêu cầu thiết bị nhận phải lưu trạng thái bộ đếm, có thể làm tăng nhu cầu bộ nhớ.

3.2.4 Phát hiện xâm nhập (Intrusion Detection System - IDS)

Nguyên lý. IDS giám sát và phân tích luồng dữ liệu trên bus CAN để phát hiện các hành vi bất thường hoặc độc hại.

Cách thực hiện.

- Thu thập dữ liệu CAN và đánh giá bất thường dựa trên tập luật (*rule-based*) hoặc mô hình học máy.
- So sánh với hành vi bình thường để phát hiện dấu hiệu lạ như tần suất gửi tăng đột biến hoặc xuất hiện ID không mong đợi.

- Gửi cảnh báo hoặc kích hoạt cơ chế chặn khi phát hiện rủi ro.

Ví dụ. Nếu một ID thường chỉ gửi 10 thông điệp mỗi phút nhưng đột nhiên phát 100 thông điệp trong vài giây, IDS có thể xem đây là dấu hiệu tấn công.

Ưu điểm. Không đòi hỏi thay đổi sâu vào giao thức CAN gốc. Có khả năng phát hiện các kiểu tấn công mới chưa được định nghĩa trước.

Nhược điểm. Có thể tạo ra cảnh báo giả (*false positive*). Yêu cầu năng lực xử lý tương đối cao nếu dùng mô hình phức tạp.

3.2.5 Phân vùng mạng (Segmentation)

Nguyên lý. Hệ thống CAN được chia thành nhiều vùng nhỏ để giới hạn phạm vi ảnh hưởng khi xảy ra sự cố bảo mật. Các vùng thường giao tiếp với nhau qua gateway hoặc firewall nội bộ.

Cách thực hiện.

- Thiết kế hệ thống thành các miền logic riêng biệt như động cơ, thân xe, phanh hoặc giải trí.
- Kiểm soát giao tiếp giữa các miền bằng gateway có chức năng xác thực, lọc và kiểm tra thông điệp.

Ví dụ. Nếu vùng giải trí bị xâm nhập, gateway có thể ngăn kẻ tấn công tác động trực tiếp đến ECU điều khiển động cơ hoặc hệ thống phanh.

Ưu điểm. Hạn chế phạm vi lây lan của tấn công khi một node bị chiếm quyền. Dễ kết hợp với các cơ chế lọc và kiểm soát truy cập.

Nhược điểm. Làm tăng độ phức tạp trong thiết kế hệ thống. Đòi hỏi thêm phần cứng hoặc phần mềm trung gian.

3.2.6 Chữ ký số (Digital Signatures)

Nguyên lý. Chữ ký số sử dụng mật mã bất đối xứng để xác thực nguồn gốc và đảm bảo tính toàn vẹn của thông điệp.

Cách thực hiện.

- Node gửi tạo chữ ký từ thông điệp và khóa bí mật của mình.
- Node nhận dùng khóa công khai tương ứng để xác minh chữ ký.

Ví dụ. Với thông điệp M , node gửi tạo chữ ký $Sig = Sign_{K_{priv}}(M)$ và truyền cùng với M . Node nhận dùng K_{pub} để kiểm tra $Verify_{K_{pub}}(M, Sig)$.

Ưu điểm. Cho khả năng xác minh mạnh và đáng tin cậy. Không cần chia sẻ cùng một khóa bí mật giữa tất cả các node.

Nhược điểm. Tốn nhiều tài nguyên và thời gian xử lý. Kích thước chữ ký thường lớn, dễ vượt giới hạn của CAN 2.0.

3.2.7 Giao thức bảo mật lớp cao (Higher-layer Security Protocols)

Nguyên lý. Các giao thức lớp ứng dụng được xây dựng để bổ sung khả năng bảo mật cho CAN mà không cần thay đổi hoàn toàn giao thức gốc.

Cách thực hiện.

- *SecOC (Secure Onboard Communication)*: Thành phần trong AUTOSAR cung cấp cơ chế xác thực và kiểm tra toàn vẹn bằng cách gắn thêm thông tin bảo mật vào thông điệp.
- *CANcrypt*: Giao thức nhẹ hỗ trợ mã hóa luồng dữ liệu, xác thực và đồng bộ khóa tạm thời phù hợp với giới hạn của CAN 2.0.

Ưu điểm. Có thể triển khai trên nền hệ thống hiện có. Cung cấp bảo mật tương đối toàn diện, bao gồm xác thực, tính toàn vẹn và tính tươi mới của thông điệp.

Nhược điểm. Yêu cầu đồng bộ trạng thái hoặc thời gian giữa các node. Tương đối phức tạp trong triển khai và bảo trì.

Chapter 4

CAN trong ESP32

4.1 Tổng quan về giao tiếp CAN trên ESP32

ESP32 là dòng vi điều khiển SoC được sử dụng rộng rãi trong các hệ thống nhúng nhờ tích hợp đồng thời năng lực xử lý, kết nối không dây và nhiều ngoại vi truyền thông. Bên cạnh UART, SPI hay I2C, ESP32 còn cung cấp bộ điều khiển CAN cổ điển do Espressif đặt tên là TWAI (*Two-Wire Automotive Interface*). Đây là nền tảng giúp ESP32 có thể tham gia trực tiếp vào một mạng CAN thử nghiệm mà không cần bổ sung thêm bộ điều khiển CAN rời.

4.1.1 Bộ điều khiển TWAI

TWAI trên ESP32 đảm nhiệm lớp điều khiển giao thức của CAN. Khối này xử lý việc đóng gói khung dữ liệu, phân xử truy cập bus, tạo CRC, xác nhận ACK, theo dõi bộ đếm lỗi và quản lý các trạng thái truyền thông. Tuy nhiên, TWAI không tạo trực tiếp tín hiệu vi sai trên đường truyền, vì vậy vẫn cần thêm một CAN transceiver ở lớp vật lý để đưa tín hiệu ra hai đường CANH và CANL.

Thuộc tính	Mô tả
Chuẩn hỗ trợ	CAN cổ điển theo ISO 11898-1
Định dạng khung	Standard Frame với ID 11 bit và Extended Frame với ID 29 bit
Dung lượng dữ liệu	Tối đa 8 byte dữ liệu trong mỗi khung
Chức năng tích hợp	Arbitration, CRC, ACK, bộ đếm lỗi, quản lý bus-off và acceptance filter
Khả năng gán chân	Cho phép ánh xạ TX và RX qua GPIO matrix của ESP32
Giới hạn	Không hỗ trợ CAN FD, do đó không sử dụng payload 64 byte

Table 4.1: Đặc điểm chính của giao tiếp TWAI trên ESP32

4.2 Kiến trúc một nút CAN dùng ESP32

Một nút CAN sử dụng ESP32 trong đề tài có thể được xem như sự kết hợp giữa khối xử lý giao thức ở bên trong vi điều khiển và khối phát thu tín hiệu ở bên ngoài. Việc tách rõ hai khối này đặc biệt quan trọng khi phân tích phần cứng, vì nhiều lỗi triển khai phát sinh từ nhầm lẫn giữa CAN controller và CAN transceiver.

4.2.1 Vai trò của bộ điều khiển CAN

TWAI đóng vai trò bộ điều khiển CAN nội bộ. Khối này quyết định định dạng khung truyền, kiểm tra tính hợp lệ của ID, DLC và cờ điều khiển, đồng thời xử lý các cơ chế cốt lõi của CAN như tranh chấp bus theo mức ưu tiên ID hay tự động phát hiện lỗi. Ở góc nhìn phần mềm, đây là nơi firmware cấu hình tốc độ bit, chế độ hoạt động và chính sách lọc khung.

4.2.2 Vai trò của bộ thu phát CAN

CAN transceiver là thành phần thuộc lớp vật lý. Nó nhận mức logic TX/RX từ ESP32 rồi biến đổi thành cặp tín hiệu vi sai CANH/CANL trên bus. Ở chiều ngược lại, transceiver lấy tín hiệu từ bus và đưa về mức logic để TWAI có thể giải mã. Nếu thiếu transceiver, ESP32 không thể nối trực tiếp vào mạng CAN thực vì không tạo ra được điện áp và đặc tính chống nhiễu cần thiết của đường truyền vi sai.

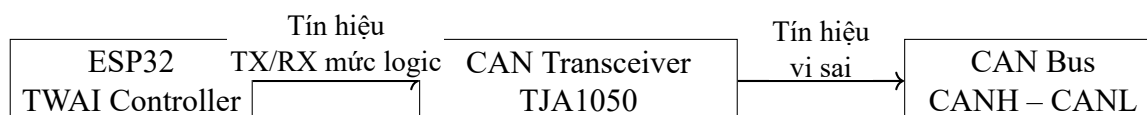


Figure 4.1: Sơ đồ khối của một nút CAN dùng ESP32

4.3 Kết nối phần cứng của mô hình

Trong mô hình thực nghiệm hiện tại, firmware mặc định sử dụng GPIO26 làm đường truyền CAN logic và GPIO27 làm đường nhận CAN logic. Hai chân này được nối với module transceiver ngoài để hình thành một nút CAN hoàn chỉnh.

4.3.1 Ảnh xạ chân tín hiệu

Nhóm tín hiệu	ESP32	Transceiver	Vai trò
Truyền dữ liệu	GPIO26	TXD	Đưa dữ liệu từ TWAI sang bộ thu phát
Nhận dữ liệu	GPIO27	RXD	Đưa dữ liệu từ bộ thu phát về TWAI
Nguồn nuôi	3.3 V hoặc 5 V	VCC	Phụ thuộc loại module transceiver sử dụng
Mass chung	GND	GND	Tạo mốc điện áp chung cho toàn nút
Đường bus	—	CANH, CANL	Kết nối ra đường truyền vì sai của mạng CAN

Table 4.2: Ảnh xạ chân CAN trong mô hình ESP32 của đề tài

4.4 Cấu trúc và nguyên lý hoạt động của TWAI

Khối TWAI trong ESP32 có thể được nhìn nhận như tập hợp của nhiều khối chức năng phối hợp với nhau để hoàn thành quá trình truyền thông CAN. Cách phân rã này giúp việc mô tả vai trò của phần cứng rõ ràng hơn và phù hợp với cách trình bày học thuật trong báo cáo.

4.4.1 Các khối chức năng chính

- Khối điều khiển chế độ hoạt động quyết định TWAI đang ở trạng thái truyền nhận bình thường, chỉ nghe hoặc kiểm thử không cần ACK.
- Khối định thời bit thiết lập tốc độ truyền và các tham số đồng bộ để mọi nút trên bus có cùng nhịp giao tiếp.
- Khối truyền chịu trách nhiệm phát khung lên bus, phối hợp với cơ chế arbitration và theo dõi việc bản tin có được xác nhận thành công hay không.
- Khối nhận lấy tín hiệu từ transceiver, giải mã thành khung CAN rồi chuyển dữ liệu cho tầng driver phần mềm.
- Khối acceptance filter loại bỏ hoặc cho phép các khung theo cấu hình tiếp nhận.
- Khối quản lý lỗi duy trì các bộ đếm lỗi, phát hiện bus-off và hỗ trợ chẩn đoán khi đường truyền gặp sự cố.

4.4.2 Quy trình khởi tạo và trao đổi dữ liệu

Ở mức tổng quát, một nút ESP32 tham gia mạng CAN thường đi qua các bước sau:

1. Chọn cặp chân GPIO dùng cho đường TX và RX của TWAI.
2. Thiết lập tốc độ bit và cấu hình chế độ hoạt động phù hợp với vai trò của nút.
3. Đặt chính sách lọc khung để xác định phạm vi bản tin cần tiếp nhận.
4. Khởi động driver TWAI và đưa transceiver vào trạng thái sẵn sàng làm việc.
5. Thực hiện truyền hoặc nhận khung thông qua hàng đợi của driver.
6. Theo dõi trạng thái lỗi để kịp thời xử lý khi bus xuất hiện bất thường.

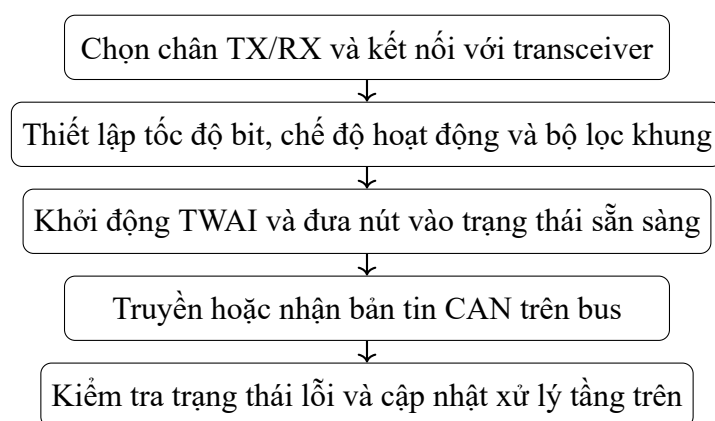


Figure 4.2: Quy trình tổng quát khi khởi tạo và vận hành TWAI trên ESP32

4.5 Các tham số cấu hình quan trọng

Để một nút ESP32 giao tiếp ổn định trên bus CAN, một số tham số cấu hình có ý nghĩa quyết định gồm chế độ hoạt động, tốc độ bit và chính sách lọc khung. Đây cũng là ba nhóm tham số ảnh hưởng trực tiếp đến việc nút được dùng như ECU mô phỏng, nút tấn công hay nút giám sát trong hệ thống CyberCAN Defense.

4.5.1 Chế độ hoạt động

- **Normal mode:** nút có thể truyền, nhận và phản hồi ACK như một thành phần hoạt động bình thường trên bus.
- **No ACK mode:** hữu ích khi kiểm thử đơn lẻ trong môi trường chưa có nút khác xác nhận bản tin, nhưng không phù hợp cho vận hành thực tế.

- **Listen Only mode:** nút chỉ nghe bus, không phát bản tin và không gửi ACK, phù hợp với vai trò giám sát hoặc IDS vì không làm thay đổi lưu lượng trên bus.

Trong phạm vi đề tài, Normal mode phù hợp với các nút đóng vai trò ECU mô phỏng hoặc nút phát tấn công, còn Listen Only mode phù hợp với các nút chuyên quan sát và thu thập log để phân tích.

4.5.2 Tốc độ bit và sự đồng bộ trên bus

Tất cả các nút trên cùng bus CAN phải sử dụng cùng tốc độ bit; nếu không, bản tin sẽ không được giải mã đúng và mạng nhanh chóng phát sinh lỗi. Một số tốc độ thường gặp được trình bày trong Bảng 4.3.

Tốc độ bit	Ý nghĩa sử dụng
125 kbps	Phù hợp với mạng tốc độ thấp hoặc đường truyền dài hơn
250 kbps	Thường dùng trong các mô hình nhúng và công nghiệp quy mô nhỏ
500 kbps	Cân bằng giữa tốc độ và độ ổn định, phù hợp với mô hình thí nghiệm của đề tài
1 Mbps	Dùng cho môi trường cự ly ngắn, yêu cầu băng thông cao hơn

Table 4.3: Một số tốc độ bit CAN thường gặp

Mô hình CyberCAN Defense lựa chọn 500 kbps vì đây là mức cấu hình phổ biến, đủ nhanh để tái hiện các tình huống tấn công như replay hay flood, đồng thời vẫn ổn định với hệ thống dây nối ngắn trên bàn thí nghiệm.

4.5.3 Bộ lọc khung và phạm vi tiếp nhận

Trên bus CAN, mọi nút đều nhìn thấy toàn bộ bản tin đang lưu thông nhưng không phải nút nào cũng cần xử lý tất cả các ID. Acceptance filter giúp giảm tải cho vi điều khiển bằng cách giới hạn những khung được đưa lên tầng phần mềm. Nếu mục tiêu là xây dựng IDS hoặc node giám sát, chính sách lọc rộng hoặc nhận toàn bộ khung là cần thiết để giữ trọn ngữ cảnh của lưu lượng. Ngược lại, với một ECU mô phỏng chỉ phục vụ số ít ID xác định, lọc chặt hơn sẽ giúp giảm số lượng bản tin không liên quan.

Vai trò nút	Định hướng cấu hình tiếp nhận
ECU mô phỏng	Ưu tiên tiếp nhận các ID phục vụ chức năng mô phỏng chính
Nút tắt công	Có thể tiếp nhận rộng để quan sát và khai thác lại lưu lượng hợp lệ
Nút giám sát hoặc IDS	Nên tiếp nhận toàn bộ hoặc gần toàn bộ khung để bảo toàn ngữ cảnh phân tích

Table 4.4: Định hướng lọc khung theo vai trò nút trong hệ thống

4.6 Truyền nhận và xử lý lỗi trên ESP32

4.6.1 Quá trình truyền và nhận khung

Ở phía truyền, firmware chuẩn bị bản tin bằng cách xác định ID, độ dài dữ liệu, kiểu khung và nội dung payload trước khi đưa vào hàng đợi của driver. Bộ điều khiển TWAI sau đó thực hiện phát bản tin lên bus và tự động tham gia cơ chế arbitration nếu cùng lúc có nhiều nút muốn truyền. Ở phía nhận, các khung hợp lệ được giải mã và chuyển vào hàng đợi nhận để phần mềm tầng trên đọc ra xử lý. Cách tổ chức này đặc biệt phù hợp với hệ thống có nhiều tác vụ đồng thời như đề tài hiện tại, nơi việc truyền bản tin, phát hiện tắt công và gửi log có thể diễn ra song song.

Trong CAN cổ điển, một khung dữ liệu có thể dùng Standard ID 11 bit hoặc Extended ID 29 bit. Việc lựa chọn kiểu ID cần được thống nhất ngay từ giai đoạn thiết kế hệ thống để tránh sai khác khi các nút giao tiếp chéo với nhau.

4.6.2 Quản lý lỗi và trạng thái bus

Một ưu điểm lớn của CAN là cơ chế phát hiện và cô lập lỗi ngay tại mức bộ điều khiển. TWAI duy trì các bộ đếm lỗi truyền và nhận; khi lỗi tăng cao, nút có thể chuyển qua các trạng thái như *error warning*, *error passive* hoặc *bus-off*. Những trạng thái này rất quan trọng trong thực nghiệm vì chúng phản ánh trực tiếp chất lượng kết nối và độ đúng của cấu hình phần cứng.

Các nguyên nhân thường gây lỗi trong mô hình ESP32 gồm:

- Sai tốc độ bit giữa các nút trên cùng bus.
- Chỉ có một nút hoạt động nên không có thiết bị ACK bản tin.
- Đấu ngược CANH và CANL hoặc không nối mass chung.
- Cấp nguồn sai cho transceiver hoặc dùng mức logic không tương thích với ESP32.

- Dây nối quá dài, tiếp xúc kém hoặc môi trường có nhiễu cao.

Trong quá trình triển khai, việc theo dõi trạng thái lỗi của TWAI giúp nhóm xác định nhanh vấn đề đang nằm ở cấu hình tốc độ, lớp vật lý hay logic xử lý tầng trên. Phân hiện thực firmware và cơ chế chẩn đoán cụ thể được trình bày ở Chương 5.

4.9 Kết luận

TWAI giúp ESP32 trở thành một nền tảng phù hợp để xây dựng các nút CAN thử nghiệm trong đề tài CyberCAN Defense. Mặc dù không hỗ trợ CAN FD và vẫn phải kết hợp với transceiver ngoài, ESP32 vẫn đáp ứng tốt yêu cầu mô phỏng ECU, thu thập lưu lượng, phát động tấn công và hỗ trợ phân tích an ninh trên bus CAN. Việc hiểu rõ kiến trúc TWAI, yêu cầu phần cứng và cơ chế xử lý lỗi là cơ sở quan trọng để triển khai đúng hệ thống được mô tả ở các chương tiếp theo.

Chapter 5

Thiết kế hệ thống

5.1 Hệ thống phần cứng

Kiến trúc phần cứng của đề tài được xây dựng theo mô hình tối giản nhưng đầy đủ để thể hiện một mạng CAN thử nghiệm có khả năng sinh tần công và phát hiện bất thường. Mỗi nút CAN gồm một vi điều khiển ESP32 kết hợp với module thu phát CAN TJA1050. Máy tính đóng vai trò điều khiển và giám sát, kết nối với ESP32 qua cổng USB–UART; ứng dụng web trên máy tính không tham gia trực tiếp vào bus vật lý mà gửi lệnh và nhận log phản hồi theo thời gian thực.

5.1.1 ESP32

ESP32 đảm nhận toàn bộ phần xử lý điều khiển của nút. Firmware được nạp qua PlatformIO với framework Arduino. Trong quá trình khởi động, ESP32 mở cổng UART ở tốc độ 115.200 bps để phục vụ terminal và ghi log, đồng thời khởi tạo bộ điều khiển CAN ở tốc độ 500 kbps.

Tham số	Giá trị	Ghi chú
Tốc độ CAN	500 kbps	Phù hợp cho mô hình thử nghiệm
Chân TX CAN	GPIO 26	Kết nối tới TXD của transceiver
Chân RX CAN	GPIO 27	Kết nối tới RXD của transceiver
Tốc độ UART	115.200 bps	Terminal và ghi log
Bảo vệ bộ đếm	Bật	Byte cuối mỗi khung là số thứ tự
Kích thước payload tối đa	7 byte	Byte thứ 8 dành cho bộ đếm
Số ID theo dõi đồng thời	16	Giới hạn bộ nhớ của cấu trúc tracker

Table 5.1: Cấu hình mặc định của firmware ESP32

Vòng lặp chính của chương trình thực hiện tuần hoàn năm công việc: đọc lệnh từ UART, thăm dò khung CAN mới từ bus, cập nhật các tác vụ tần công định kỳ, điều khiển LED báo hoạt động và phục vụ dấu nhắc terminal. Nhờ cách tổ chức vòng lặp đơn luồng này, ESP32 vừa đóng vai trò nút truyền thông CAN vừa là bộ xử lý cục bộ cho các thuật toán phát hiện xâm nhập.

5.1.2 Module CAN TJA1050

TJA1050 là IC thu phát CAN của NXP Semiconductors, đảm nhận lớp vật lý theo chuẩn ISO 11898-2. Trong mô hình của đề tài, TJA1050 thường được tích hợp sẵn trên một board nhỏ có kèm điện trở kéo và tụ lọc. Phần tử này nhận tín hiệu logic từ GPIO của ESP32, biến đổi thành cặp tín hiệu vi sai CANH/CANL và ngược lại. Các thông số kỹ thuật chính được trình bày trong Bảng 5.2.

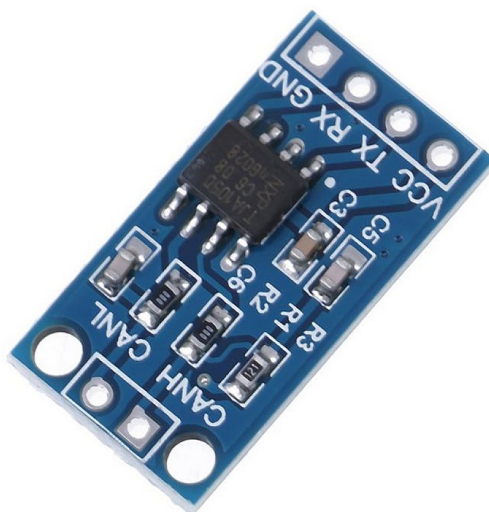


Figure 5.1: Module CAN TJA1050

Thuộc tính	Mô tả
Chuẩn giao tiếp	ISO 11898, CAN 2.0A/B
Điện áp cấp nguồn	4,75 V – 5,25 V (danh nghĩa 5 V)
Mức logic đầu vào	Tương thích 3,3 V và 5 V
Tốc độ tối đa	1 Mbps
Chân kết nối chính	TXD, RXD, VCC, GND, CANH, CANL
Điện trở kết thúc bus	120 Ω tại mỗi đầu bus

Table 5.2: Đặc điểm kỹ thuật của module TJA1050

Một lưu ý quan trọng khi ghép ESP32 (3,3 V) với TJA1050 (5 V) là mức điện áp ngõ ra của transceiver trả về GPIO của ESP32 có thể ở mức 5 V tùy phiên bản module. Trong trường hợp này, cần sử dụng phân áp điện trở hoặc IC dịch mức logic để bảo vệ vi điều khiển.

5.1.3 Sơ đồ kết nối phần cứng

Sơ đồ kết nối giữa ESP32 và module TJA1050 được thể hiện trong Hình 5.1. Chân GPIO 26 của ESP32 làm đường truyền CAN logic, chân GPIO 27 làm đường nhận; hai chân này nối trực tiếp tới các ngõ vào/ra tương ứng của transceiver. Từ transceiver, cặp đường CANH/CANL đi ra bus chung



Figure 5.2: Sơ đồ khối phần cứng tổng quan

5.2 Hệ thống phần mềm

Hệ thống phần mềm được chia thành hai lớp phối hợp qua giao tiếp UART và HTTP/WebSocket. Lớp thứ nhất là firmware nhúng chạy trực tiếp trên ESP32, chịu trách nhiệm giao tiếp CAN, mô phỏng tấn công và phát hiện bất thường. Lớp thứ hai là ứng dụng web chạy trên máy tính, đóng vai trò giao diện điều khiển và giám sát thời gian thực. Kiến trúc tổng thể được mô tả trong Hình 5.3.

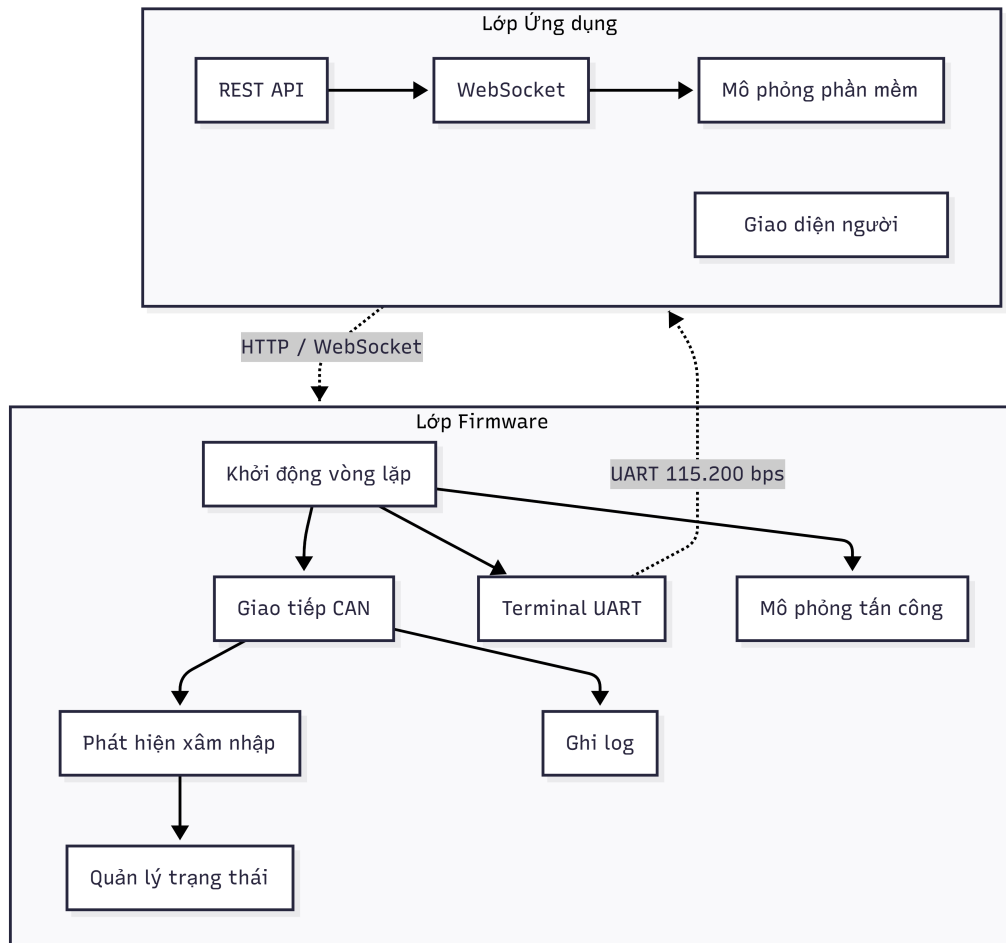


Figure 5.3: Kiến trúc tổng thể hai lớp phần mềm của hệ thống

5.2.1 Firmware ESP32

Firmware được tổ chức thành bảy mô-đun, mỗi mô-đun phụ trách một nhóm chức năng độc lập. Các mô-đun giao tiếp với nhau qua một tập hợp biến trạng thái chia sẻ, cho phép toàn bộ hệ thống hoạt động trong một luồng đơn mà không cần cơ chế đồng bộ phức tạp.

Mô-đun	Vai trò
Khởi động và vòng lặp chính	Điều phối toàn bộ luồng xử lý của firmware
Giao tiếp CAN	Khởi tạo bus, gửi và nhận khung, bảo vệ bộ đếm
Xử lý lệnh terminal	Phân tích lệnh UART, in trạng thái hệ thống
Mô phỏng tấn công	Quản lý và thực thi năm loại tác vụ tấn công định kỳ
Phát hiện xâm nhập	Phân tích hành vi khung, sinh cảnh báo, quản lý luật chặn
Ghi log	Định dạng và xuất thông điệp ra UART theo chuẩn thống nhất
Quản lý trạng thái	Lưu trữ toàn bộ dữ liệu vận hành trong RAM nội bộ

Table 5.3: Các mô-đun firmware và vai trò

Các kiểu dữ liệu trung tâm được khai báo tập trung trong một tệp tiêu đề chung, bao gồm cấu trúc khung CAN, bản ghi cảnh báo, khung được bắt gần nhất, các tác vụ tấn công, bộ theo dõi từng ID và các cấu trúc phòng thủ. Nhờ đó, toàn bộ mô-đun chia sẻ cùng một biểu diễn dữ liệu và giảm sự phụ thuộc lẫn nhau.

5.2.2 Giao diện CAN và cơ chế bảo vệ bộ đếm

Mọi khung do người dùng tạo ra đều được kiểm tra và chuẩn hóa trước khi phát lên bus. Quá trình này bao gồm: xác nhận CAN ID nằm trong phạm vi hợp lệ, kiểm tra độ dài dữ liệu, và tự động chèn thêm một byte số thứ tự tăng dần vào cuối payload. Nhờ đó, mỗi khung hợp lệ trên bus luôn mang một bộ đếm phân biệt giúp phát hiện hành vi phát lại.

Ở phía nhận, mô-đun giao tiếp CAN đọc khung từ bus theo từng lô nhỏ. Mỗi khung nhận được lần lượt qua các bước xử lý: kiểm tra xem ID có đang bị chặn bởi cơ chế phòng thủ hay không, ghi log, phân tích bởi hệ thống phát hiện xâm nhập và lưu lại làm khung bắt gần nhất để phục vụ tính năng replay.

5.2.3 Hệ thống phát hiện xâm nhập (IDS)

Hệ thống phát hiện xâm nhập hoạt động theo hai cấp độ theo dõi song song. Ở cấp độ đầu tiên, mỗi CAN ID trên bus được theo dõi riêng biệt bởi một bộ theo dõi (tracker) lưu trữ lịch sử về tần suất xuất hiện, nội dung payload, bộ đếm thứ tự và đường cơ sở bình thường. Hệ thống duy trì đồng thời tối đa 16 bộ theo dõi; khi số ID vượt quá giới hạn, bộ theo dõi của ID ít hoạt động nhất sẽ được tái sử dụng.

Ở cấp độ thứ hai, một bộ theo dõi lưu lượng toàn cục quan sát hành vi tổng thể của toàn bộ bus trong một cửa sổ thời gian trượt, ghi nhận số lượng ID mới và mức độ thay đổi ID liên tiếp – hai chỉ số đặc trưng của tấn công Fuzzing.

Loại cảnh báo	Phương pháp phát hiện	Điều kiện kích hoạt
Replay (phát lại)	So sánh bộ đếm	Byte số thứ tự không tăng đúng thứ tự
DoS (ngập lưu lượng)	Ước tính tốc độ	Tốc độ cùng ID > 60 khung/giây
Fuzzing (quét ngẫu nhiên)	Theo dõi toàn bus	≥ 4 ID mới, ≥ 5 đổi ID / 1,2 giây
Spoofing (giả mạo)	So sánh baseline	Payload lệch ≥ 2 byte so với mẫu bình thường

Table 5.4: Tổng hợp các loại cảnh báo IDS và điều kiện kích hoạt

5.2.4 Bộ mô phỏng tấn công

Bộ mô phỏng tấn công cung cấp năm kiểu tấn công có thể kích hoạt độc lập qua terminal UART hoặc giao diện web. Mỗi kiểu tấn công chạy như một tác vụ định kỳ trong vòng lặp chính với chu kỳ có thể tùy chỉnh.

Kiểu tấn công	Chu kỳ mặc định	Mô tả
Replay	250 ms	Phát lại nguyên bản khung hợp lệ đã bắt được, kể cả bộ đếm cũ
DoS	20 ms	Ngập lưu lượng trên một CAN ID với tần suất cao
Fuzzing	35 ms	Phát liên tục các khung với ID và payload ngẫu nhiên
Spoofing	120 ms	Giả mạo ID của bản tin hợp lệ với nội dung bị biến đổi

Table 5.5: Các kiểu tấn công được hỗ trợ

Tấn công Replay có đặc điểm nổi bật là phát lại khung ở mức thô (wire level), tức là không qua bước chuẩn hóa và bổ sung bộ đếm mới. Điều này khiến byte số thứ tự bị lặp lại, tạo điều kiện cho IDS phát hiện. Tấn công Fuzzing sử dụng bộ sinh số giả ngẫu nhiên để tạo ra các CAN ID và payload biến đổi liên tục trong mỗi chu kỳ. Tấn công Spoofing dùng ID của một bản tin hợp lệ đã biết nhưng biến đổi nội dung theo một mẫu xoay vòng qua từng chu kỳ phát.

5.2.5 Ứng dụng web giám sát và điều khiển

Ứng dụng web sử dụng FastAPI làm backend và HTML/CSS/JavaScript thuần túy làm giao diện người dùng. Hệ thống được khởi động dưới dạng máy chủ web tại cổng 8000, phục vụ đồng thời giao diện HTML và luồng dữ liệu thời gian thực qua WebSocket.

Backend phân biệt hai chế độ kết nối nút. Trong chế độ phần cứng thực, backend chuyển tiếp lệnh văn bản tới firmware qua cổng nối tiếp. Trong chế độ mô phỏng phần mềm, backend tự sinh luồng sự kiện tấn công và phản hồi IDS mà không cần phần cứng thực, phục vụ cho việc trình diễn giao diện. Giao diện web cung cấp đầy đủ chức năng thêm/xóa nút, gửi khung CAN, kích hoạt từng kiểu tấn công và bật/tắt cơ chế phòng thủ. Toàn bộ log và sự kiện được phát trực tiếp tới trình duyệt theo thời gian thực qua kênh WebSocket.

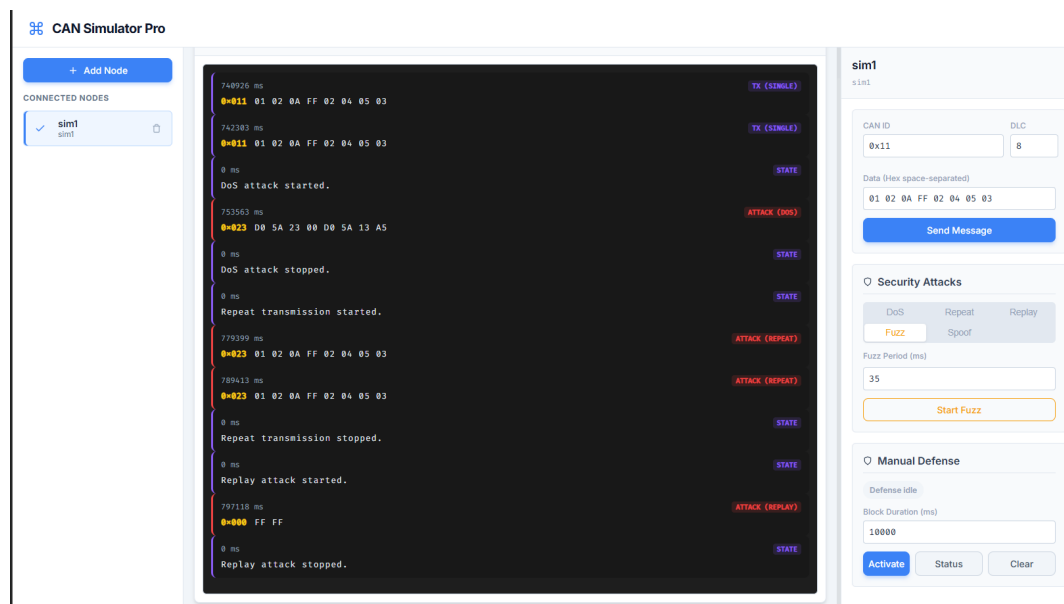


Figure 5.4: Giao diện web giám sát, tấn công và phòng thủ CAN

5.2.6 Quản lý trạng thái và lưu trữ dữ liệu

Hệ thống không sử dụng cơ sở dữ liệu lưu trữ bền vững. Phía firmware, toàn bộ trạng thái vận hành được lưu trong bộ nhớ RAM nội bộ của ESP32, bao gồm khung bắt gói nhất, bản ghi cảnh báo, các tác vụ tấn công đang chạy và bộ nhớ phòng thủ. Dung lượng sử dụng nằm trong giới hạn an toàn của ESP32 với 520 kB SRAM tích hợp.

Phía ứng dụng web, trạng thái của tất cả nút và lịch sử khung quan sát được cũng chỉ lưu trong bộ nhớ của tiến trình Python. Khi chương trình dừng, toàn bộ thông tin này bị giải phóng. Thiết kế này phù hợp với mục tiêu trình diễn và thực nghiệm ngắn hạn, nhưng cần bổ sung lớp lưu trữ bền vững nếu mở rộng sang các ứng dụng giám sát dài hạn.

5.3 Nguyên lý hoạt động của hệ thống

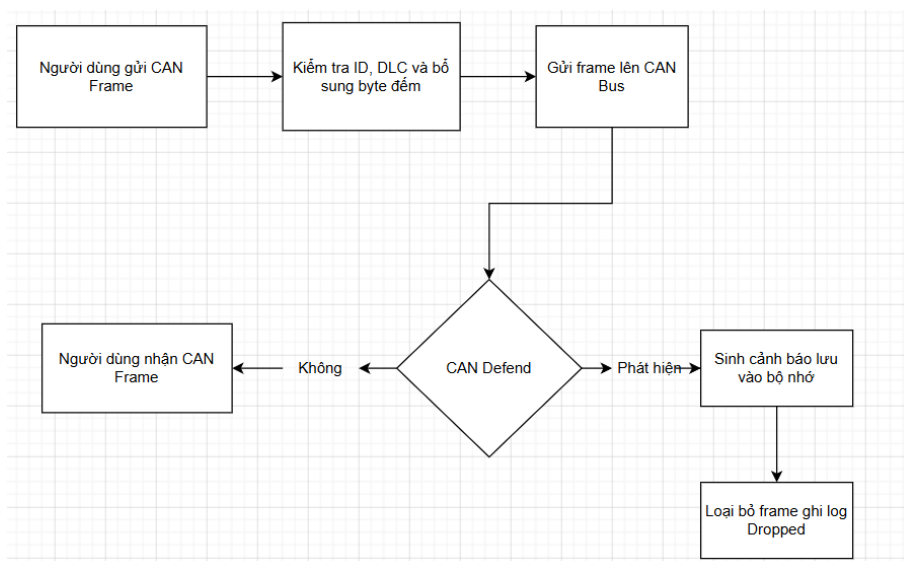


Figure 5.5: Sơ đồ luồng xử lý tổng thể của hệ thống

5.3.1 Giao tiếp CAN cơ bản

Ở chế độ bình thường, người dùng gửi một khung CAN đơn bằng lệnh `send <id> <dlc> <data>` qua terminal. Firmware kiểm tra tính hợp lệ của các tham số, bổ sung byte số thứ tự vào cuối payload rồi phát lên bus. Khi nhận được khung từ bus, nút ghi log theo định dạng chuẩn có nhãn thời gian, chiều truyền, CAN ID, độ dài dữ liệu và nội dung. Đồng thời khung được lưu lại làm dữ liệu nền cho tính năng phát lại. LED trên bo mạch nhấp nháy ngắn để xác nhận mỗi sự kiện truyền hoặc nhận thành công.

5.3.2 Cơ chế bảo vệ bộ đếm thông điệp

Khi tính năng bảo vệ bộ đếm được bật, mỗi khung gửi đi đều có thêm một byte số thứ tự tăng dần ở vị trí byte cuối cùng của payload, làm tổng độ dài tăng lên một byte. Bộ đếm là số nguyên 8-bit không dấu, tăng liên tục và quay về 0 sau giá trị 255. Phía nhận đọc byte cuối của mỗi khung đến, đối chiếu với giá trị kỳ vọng dựa trên lần nhận trước của cùng CAN ID. Bất kỳ sai lệch nào đều bị ghi nhận là dấu hiệu bất thường và sinh cảnh báo phát lại loại đếm ngược.

5.3.3 Replay Attack

- **Mô hình tấn công:** Tấn công phát lại (Replay Attack) xảy ra khi một thực thể trái phép tiến hành rình rập (sniffing) và sao chép các thông điệp CAN hợp lệ đang truyền tải trên bus. Sau

đó, thực thể này tái phát (inject) các thông điệp nguyên bản đó vào mạng tại một thời điểm khác.

- **Hậu quả:** Các Bộ điều khiển điện tử (ECU) sẽ tiếp nhận và thực thi các lệnh bị lặp lại (ví dụ: kích hoạt hệ thống phanh, vô hiệu hóa khóa cửa) do giao thức CAN nguyên thủy thiếu vẹn toàn về nhãn thời gian. Điều này dẫn đến sự sai lệch nghiêm trọng trong trạng thái vận hành của phương tiện.
- **Cơ chế phòng thủ:** Giải pháp lý thuyết tối ưu để ngăn chặn tấn công phát lại là tích hợp các định danh động như bộ đếm tăng dần (Monotonic Counter) hoặc Mã xác thực thông điệp (MAC) vào dữ liệu tải. Qua đó, hệ thống phát hiện xâm nhập (IDS) có thể kiểm chứng tính mới (freshness) của dữ liệu và loại bỏ các gói tin lỗi thời.

5.3.4 Spoofing Attack

- **Mô hình tấn công:** Tấn công giả mạo (Spoofing) là kỹ thuật đánh lừa định tuyến, trong đó kẻ tấn công phát đi các khung dữ liệu mang CAN ID hợp lệ nhưng cố ý ngụy tạo cấu trúc hoặc giá trị của vùng dữ liệu tải (payload) nhằm đánh lừa các ECU đích.
- **Hậu quả:** Việc tiêm các thông số giả mạo (ví dụ: tốc độ vòng tua động cơ, nhiệt độ cảm biến) sẽ làm nhiễu loạn logic điều khiển của hệ thống. Nếu các ECU đưa ra quyết định dựa trên luồng dữ liệu giả mạo này, sự an toàn cơ học của phương tiện có thể bị đe dọa trực tiếp.
- **Cơ chế phòng thủ:** Phương thức phòng vệ nền tảng là xây dựng mô hình hành vi bình thường (baseline) cho mạng CAN dựa trên chu kỳ phát và đặc trưng tải trọng của từng định danh. Mọi sự sai lệch bất thường về độ dài dữ liệu (DLC) hoặc giá trị payload lệch khỏi quỹ đạo thống kê đều bị IDS phân loại là hành vi giả mạo.

5.3.5 DoS Attack

- **Mô hình tấn công:** Tấn công từ chối dịch vụ (Denial of Service - DoS) được thực thi thông qua việc bơm tràn ngập (flooding) mạng CAN bằng các khung dữ liệu mang định danh có mức độ ưu tiên cao nhất (CAN ID mang giá trị thấp) với tần suất cực đại.
- **Hậu quả:** Do cơ chế phân xử tranh chấp dựa trên mức ưu tiên (Arbitration) của giao thức CAN, các thông điệp từ các ECU hợp lệ sẽ liên tục bị từ chối quyền truy cập bus. Trạng thái này gây ra độ trễ nghiêm trọng hoặc làm tê liệt hoàn toàn khả năng trao đổi thông tin của toàn mạng.
- **Cơ chế phòng thủ:** Cơ sở lý luận để giảm thiểu DoS là giám sát và phân tích lưu lượng dựa trên tần suất (Rate-based Intrusion Detection). Hệ thống IDS cần tính toán mật độ thông điệp

trong các cửa sổ thời gian (time windows). Bất cứ nút nào phát sinh lưu lượng vượt quá giới hạn băng thông lý thuyết của mạng sẽ bị nhận diện và cách ly.

5.3.6 Fuzzing Attack

- **Mô hình tấn công:** Quét ngẫu nhiên (Fuzzing) là chiến lược tấn công diện rộng, trong đó kẻ tấn công liên tục tiêm vào bus hàng loạt khung CAN với các định danh (ID) và vùng dữ liệu tải (payload) được sinh ngẫu nhiên hoàn toàn sau mỗi chu kỳ.
- **Hậu quả:** Cuộc tấn công này đẩy các ECU vào trạng thái phải liên tục xử lý một khối lượng khổng lồ các thông điệp rác không xác định. Việc này không chỉ gây lãng phí tài nguyên xử lý mà còn có thể vô tình kích hoạt các chức năng ẩn (diagnostic messages), dẫn đến hệ thống hoạt động hỗn loạn.
- **Cơ chế phòng thủ:** Khác với việc giám sát theo từng ID đơn lẻ, nguyên lý nhận diện Fuzzing dựa trên phép phân tích tính biến động toàn cục của bus. Sự xuất hiện ồ ạt của các ID ngoại lai hoặc tỷ lệ luân chuyển ID quá cao trong một đơn vị thời gian là các đặc trưng dị thường giúp IDS kích hoạt trạng thái phòng vệ.

5.3.7 Cơ chế phòng thủ chủ động

Sau khi IDS phát sinh cảnh báo, người vận hành có thể kích hoạt phòng thủ qua lệnh terminal hoặc nút tương ứng trên giao diện web. Hệ thống đọc thông tin từ cảnh báo gần nhất và thiết lập quy tắc chặn phù hợp với từng loại tấn công. Thời gian duy trì luật chặn mặc định là 10 giây nhưng có thể tùy chỉnh.

Loại tấn công	Đối tượng bị chặn	Tiêu chí lọc
Replay (phát lại)	Khung cùng ID và nội dung	Khớp chính xác với chữ ký khung bị cảnh báo
DoS (ngập lưu lượng)	Mọi khung cùng ID	Chặn toàn bộ, không phân biệt nội dung
Spoofing (giả mạo)	Khung cùng ID đáng ngờ	Lọc theo độ lệch so với baseline đã học
Fuzzing (quét ngẫu nhiên)	Mọi ID chưa tin cậy	Chỉ cho phép ID đã ổn định trước cảnh báo

Table 5.6: Chiến lược phòng thủ theo từng loại tấn công

Cần lưu ý rằng cơ chế phòng thủ hiện tại hoạt động ở tầng phần mềm của nút nhận, không loại bỏ khung tấn công khỏi bus vật lý. Kẻ tấn công vẫn có thể tiếp tục phát khung; hệ thống chỉ ngăn tầng ứng dụng xử lý các khung bị nghi ngờ trong thời gian áp dụng luật chặn. Đây là

hạn chế vốn có của kiến trúc CAN không có cơ chế xác thực ở lớp liên kết dữ liệu và là hướng mở rộng tiếp theo của đề tài.

Chapter 6

Thực thi, đo lường và đánh giá

I. Giao tiếp 2 chiều

Trong giai đoạn giao tiếp 2 chiều, hệ thống được vận hành với hai nút ESP32 tham gia cùng một bus CAN để kiểm tra khả năng gửi và nhận bản tin theo cả hai chiều. Mục tiêu của phần này là chứng minh rằng mô hình phần cứng và firmware hiện tại có thể duy trì trao đổi dữ liệu ổn định trước khi đưa thêm các kịch bản tấn công và cơ chế phòng thủ.

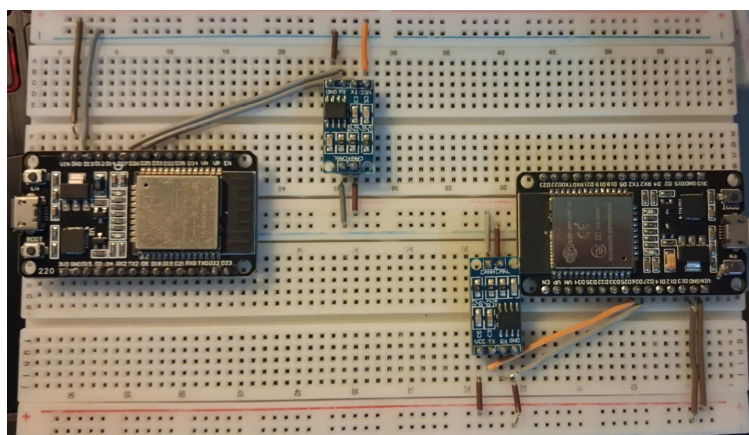


Figure 6.1: Mô hình phần cứng giao tiếp CAN với 2 node ESP32

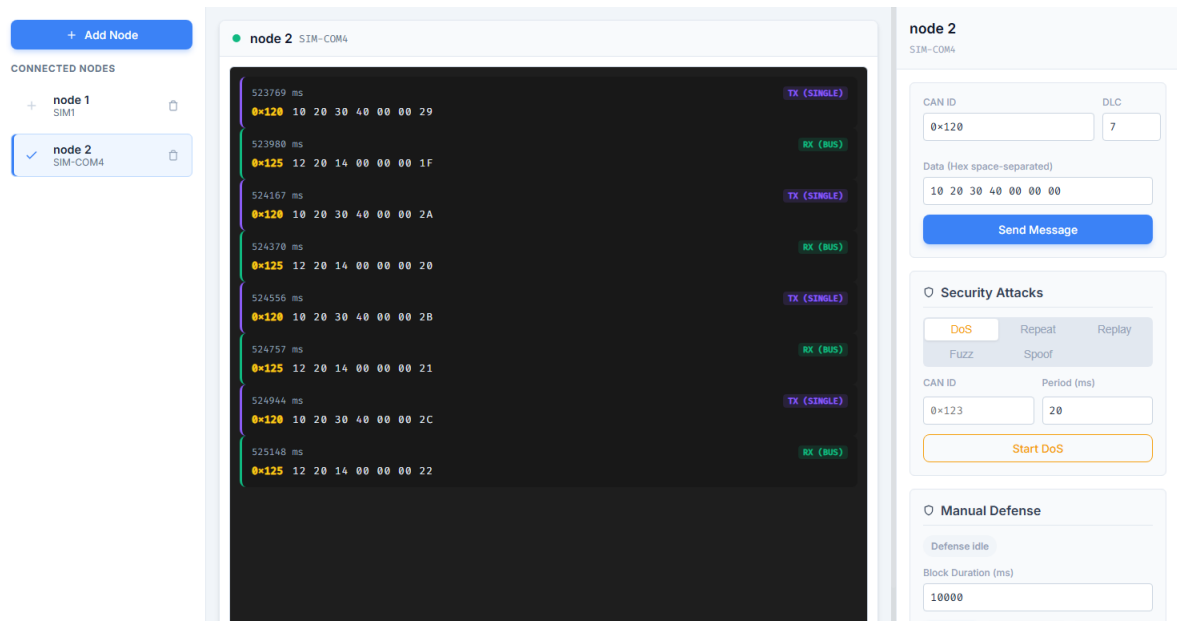


Figure 6.2: Phiên giao tiếp CAN hai chiều bình thường hiển thị trên giao diện web

Ảnh chụp ở Hình 6.2 cho thấy nút node 2 vừa phát khung CAN ID 0x120 vừa nhận khung ID 0x125 từ nút còn lại trên cùng bus. Các bản tin TX và RX xuất hiện xen kẽ, đồng thời byte cuối của dữ liệu thay đổi tuần tự theo từng lần gửi.

II. Các kịch bản tấn công và phản ứng phòng thủ

Để mô phỏng tấn công trên mạng CAN, hệ thống bổ sung thêm một nút thứ ba đóng vai trò thiết bị tấn công. Nút này có nhiệm vụ quan sát lưu lượng hợp lệ trên bus, lưu lại bản tin đã thu được và phát lại hoặc phát dòn với chu kỳ xác định nhằm kiểm tra khả năng chịu lỗi của hệ thống truyền nhận.

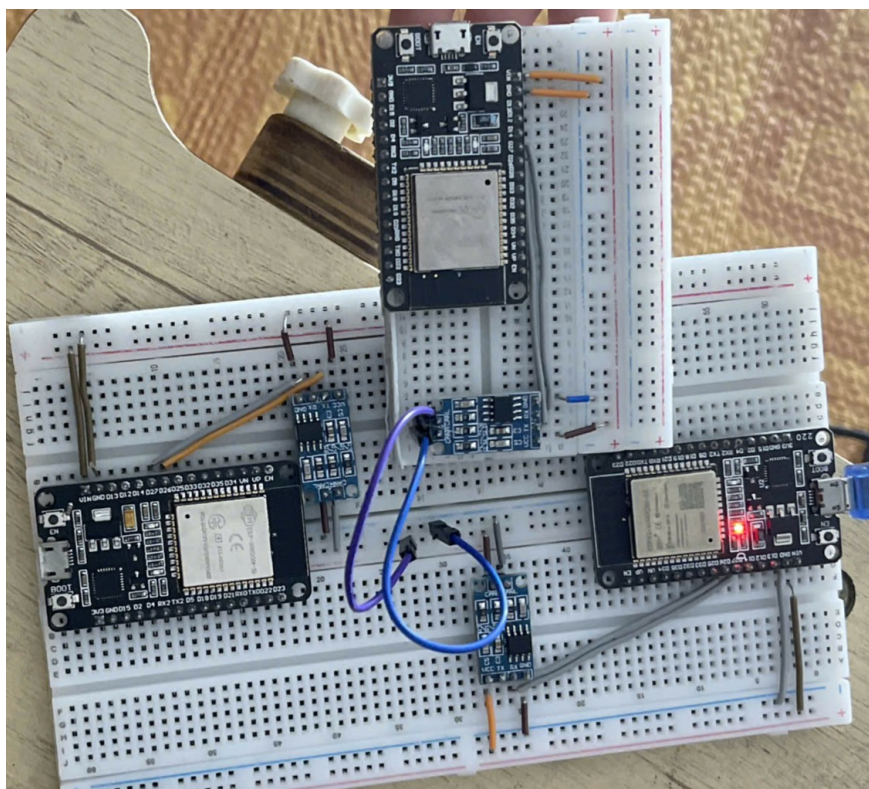


Figure 6.3: Mô hình phần cứng thực tế khi bổ sung thêm node hacker vào bus CAN

Trên giao diện điều khiển, người vận hành có thể chọn nút đang thao tác, cấu hình tham số tấn công và theo dõi phản hồi của từng nút theo thời gian thực. Để phần đánh giá rõ ràng hơn, chương này trình bày theo từng kịch bản riêng gồm DoS, Replay, Fuzzing và Spoofing. Trong mỗi kịch bản, kết quả được mô tả theo hai bước liên tiếp: bước hệ thống sinh cảnh báo ALERT và bước người vận hành kích hoạt defense để chặn hoặc giảm tác động của lưu lượng bất thường lên nút nhận.

1. Tấn công DoS

Trong kịch bản DoS, nút tấn công phát liên tiếp nhiều khung CAN cùng một ID với chu kỳ rất ngắn để làm tăng nhanh mật độ lưu lượng trên bus. Mục tiêu của phần này là kiểm tra khả năng quan sát hiện tượng ngập bản tin ở nút nhận và khả năng sinh cảnh báo khi tốc độ khung vượt ngưỡng bình thường.

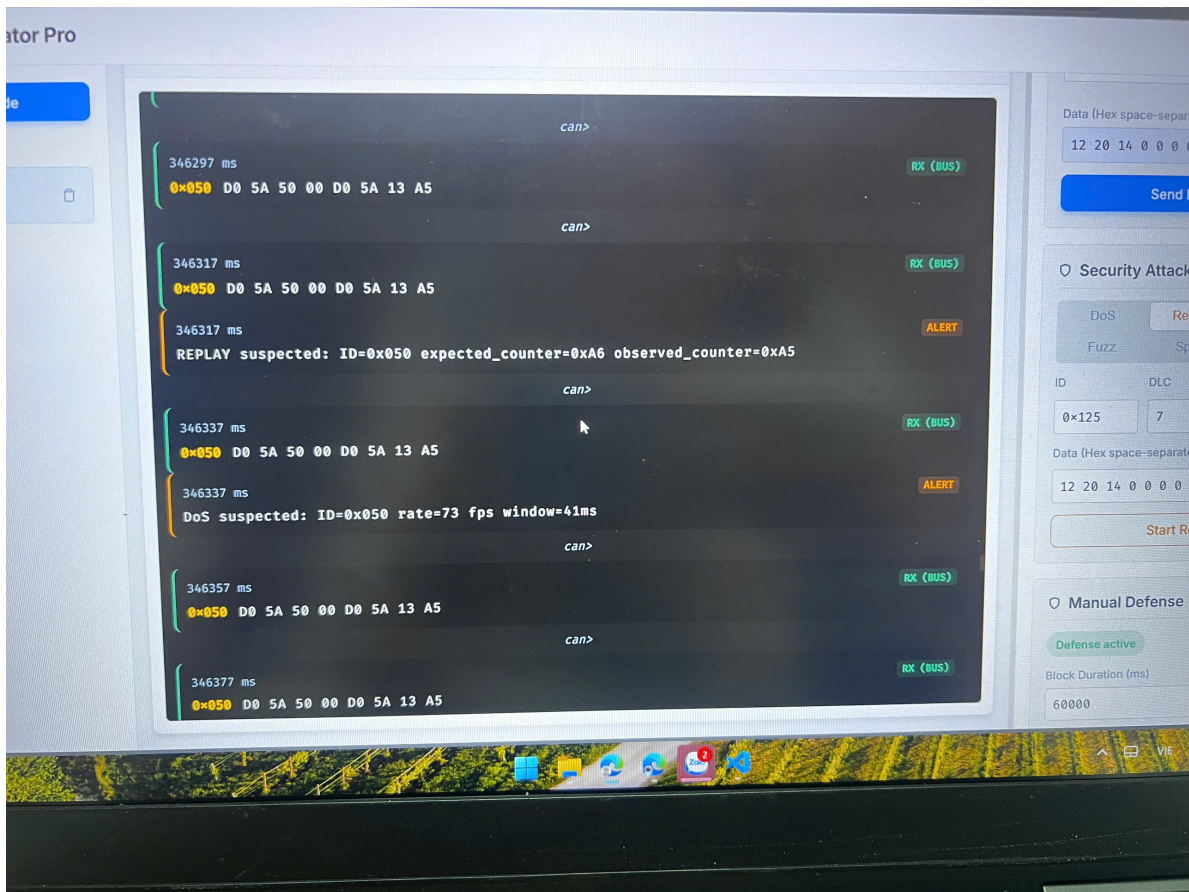


Figure 6.4: Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công DoS

Khi DoS diễn ra, giao diện phía nút nhận thường xuất hiện dày đặc các bản tin RX cùng ID, trong khi phân cảnh báo có thể hiển thị trạng thái DoS suspected. Trên bus, hiện tượng này tương ứng với việc số khung hợp lệ tăng mạnh trong một cửa sổ thời gian ngắn.

Bước cảnh báo: Ở Hình 6.4, các khung RX với ID 0x050 xuất hiện liên tiếp trên giao diện, đồng thời hệ thống sinh ra nhãn ALERT với dòng DoS suspected. Điều này cho thấy bộ phát hiện đã nhận ra tốc độ khung tăng bất thường và đánh dấu đây là một phiên có dấu hiệu ngập lưu lượng. Trong ảnh còn có thêm một dòng REPLAY suspected xuất hiện trước đó, nhưng trọng tâm của bước này là cảnh báo DoS suspected dùng để kích hoạt cơ chế phòng thủ ở bước tiếp theo.

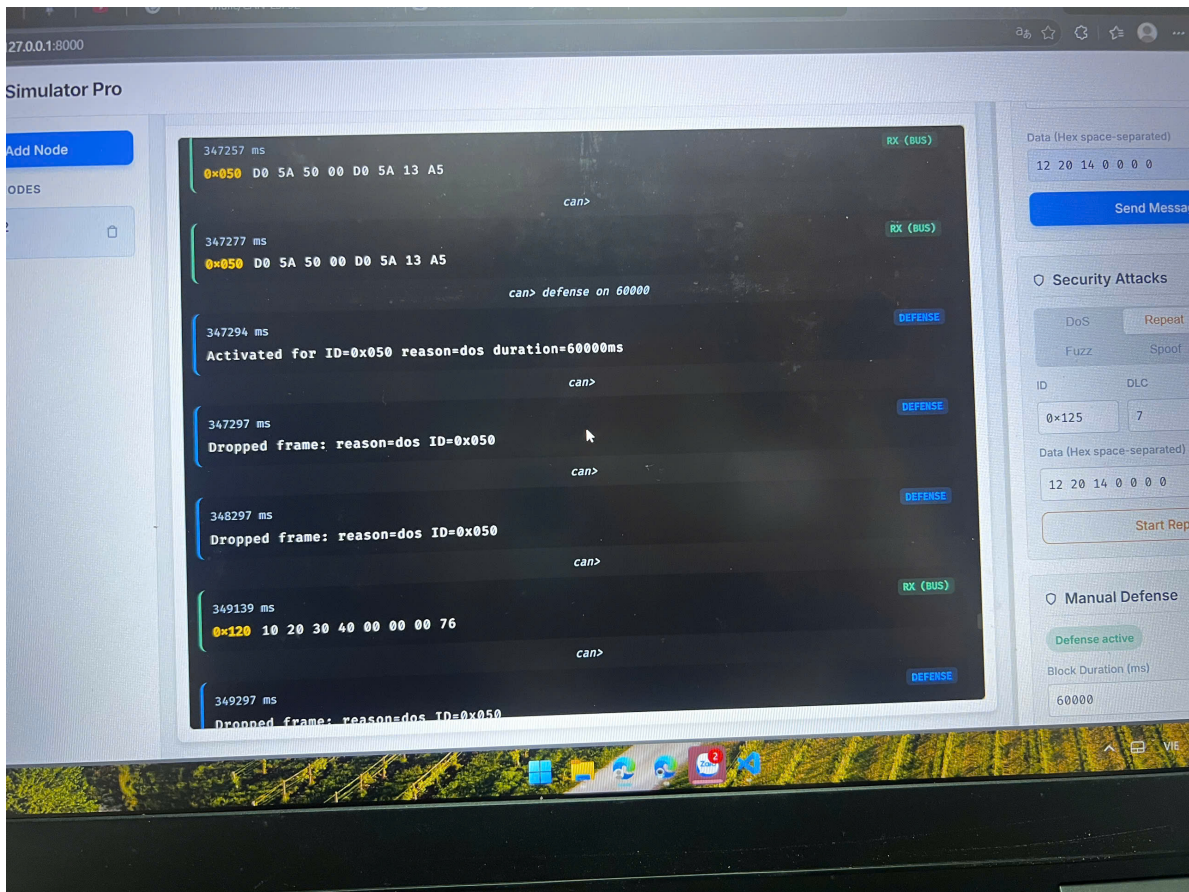


Figure 6.5: Giao diện sau khi bật defense để chặn lưu lượng DoS

Bước phòng thủ: Ở Hình 6.5, sau khi người vận hành gửi lệnh `defense on 60000`, hệ thống xác nhận kích hoạt phòng thủ bằng thông báo `Activated for ID=0x050 reason=dos`. Ngay sau đó, các khung tiếp theo thuộc ID nghi ngờ không còn được xử lý như dữ liệu hợp lệ mà bị ghi nhận dưới dạng `Dropped frame`. Kết quả này cho thấy cơ chế *defend* không làm mất hoàn toàn lưu lượng trên bus vật lý, nhưng đã chặn được tác động của luồng DoS lên tầng ứng dụng ở nút nhận.

2. Tấn công Replay

Kịch bản Replay yêu cầu nút tấn công đã quan sát được ít nhất một khung hợp lệ trên bus trước khi phát lại. Sau khi lệnh `replay start` được kích hoạt, khung vừa thu sẽ được gửi lặp theo chu kỳ định sẵn, nhằm kiểm tra khả năng phát hiện khung lặp của hệ thống.

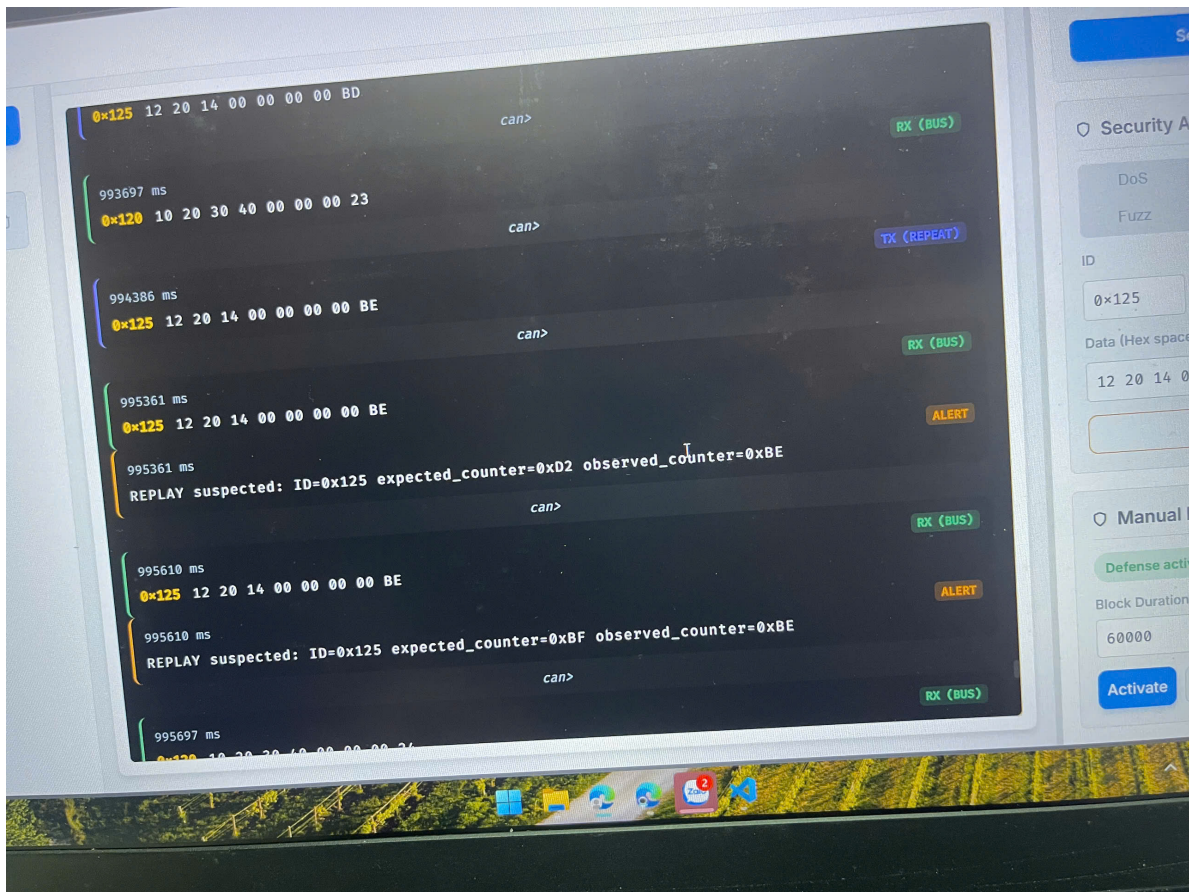


Figure 6.6: Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Replay

Nếu hệ thống có chèn bộ đếm ở byte cuối, khung bị phát lại sẽ giữ nguyên giá trị bộ đếm cũ. Điều này giúp giải thích vì sao phía nhận có thể sinh cảnh báo REPLAY suspected do trùng lặp dữ liệu hoặc do sai lệch chuỗi bộ đếm giữa các lần xuất hiện của cùng một CAN ID.

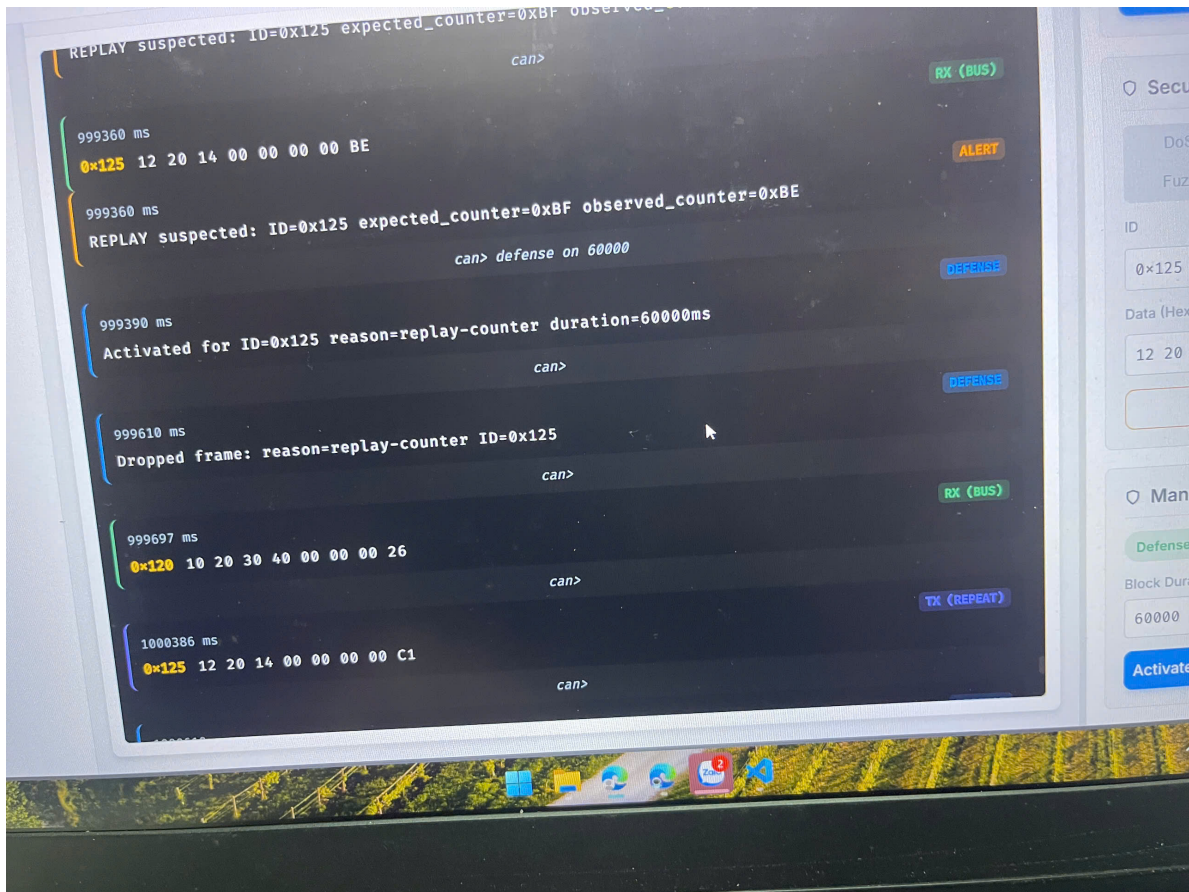


Figure 6.7: Giao diện sau khi bật defense để chặn khung replay

Bước cảnh báo: Ở Hình 6.6, hệ thống tại nút nhận liên tục quan sát thấy các khung mang ID 0x125 có giá trị bộ đếm cũ bị lặp lại. Hệ thống tiến hành so sánh bộ đếm quan sát được với bộ đếm kỳ vọng và sinh ra cảnh báo `REPLAY suspected`, ví dụ `expected_counter=0xD2` `observed_counter=0xBE`. Điều này minh chứng rằng cùng một khung đã bị phát lại thay vì tiếp tục tăng tuần tự như trong phiên truyền hợp lệ.

Bước phòng thủ: Ở Hình 6.7, sau khi người vận hành gửi lệnh `defense on 60000`, hệ thống xác nhận kích hoạt phòng thủ bằng thông báo `Activated for ID=0x125 reason=replay-counter duration=60000ms`. Ngay sau đó, các khung replay tiếp theo không còn được xử lý như dữ liệu hợp lệ mà bị ghi nhận dưới dạng `Dropped frame: reason=replay-counter ID=0x125`. Cơ chế này giúp chặn luồng phát lại ở tầng ứng dụng trong khi việc giám sát bus vẫn tiếp tục diễn ra.

3. Tấn công Fuzzing

Trong kịch bản Fuzzing, nút tấn công liên tục thay đổi ID và dữ liệu tải để tạo ra chuỗi bản tin bất thường, khó dự đoán. Mục tiêu của phần này là đánh giá khả năng phát hiện sự thay đổi nhanh về mẫu lưu lượng, đặc biệt khi số lượng ID khác nhau tăng lên trong thời gian ngắn.

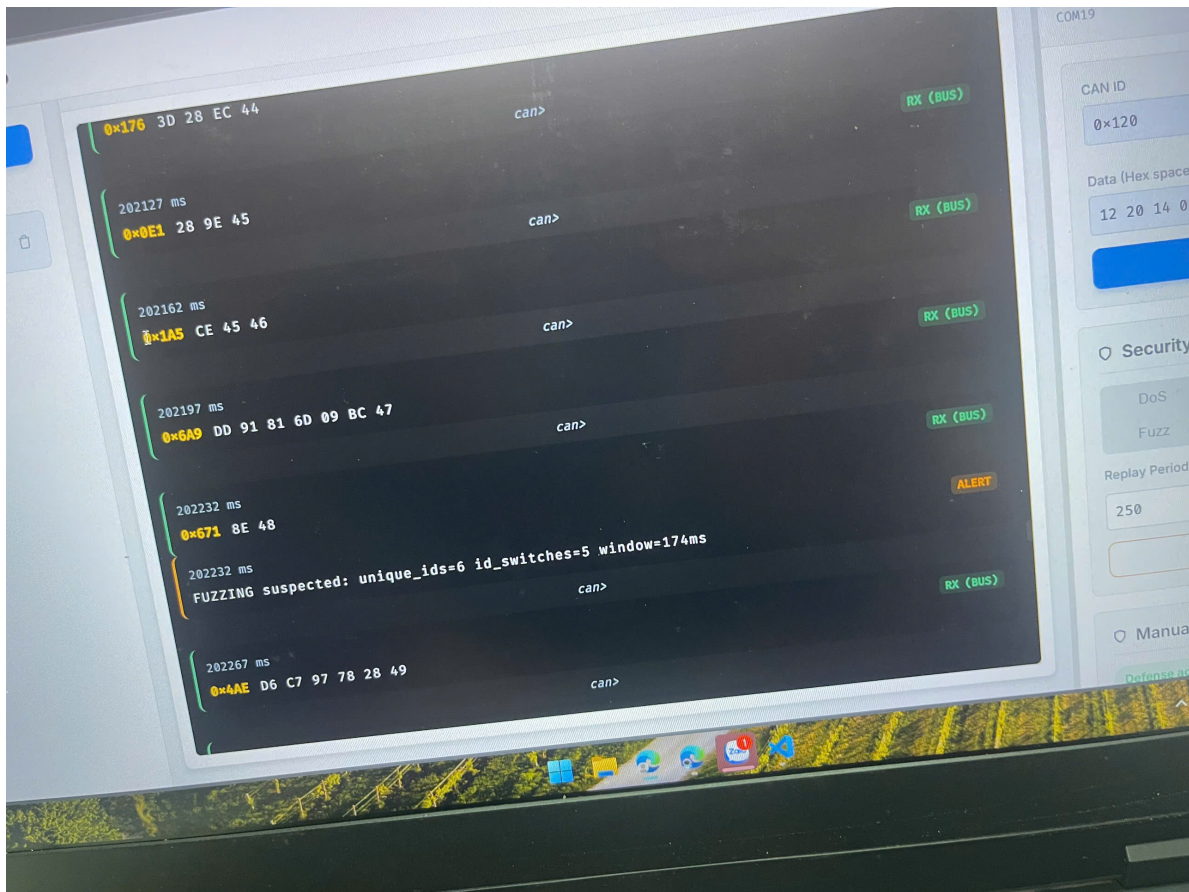


Figure 6.8: Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Fuzzing

Khi quan sát ở nút nhận, các bản tin xuất hiện với ID thay đổi nhanh và nội dung dữ liệu không ổn định như trong chế độ truyền bình thường. Trong log, hệ thống có thể sinh cảnh báo FUZZING suspected khi số lượng ID khác nhau và tần suất chuyển đổi ID vượt mức cho phép.

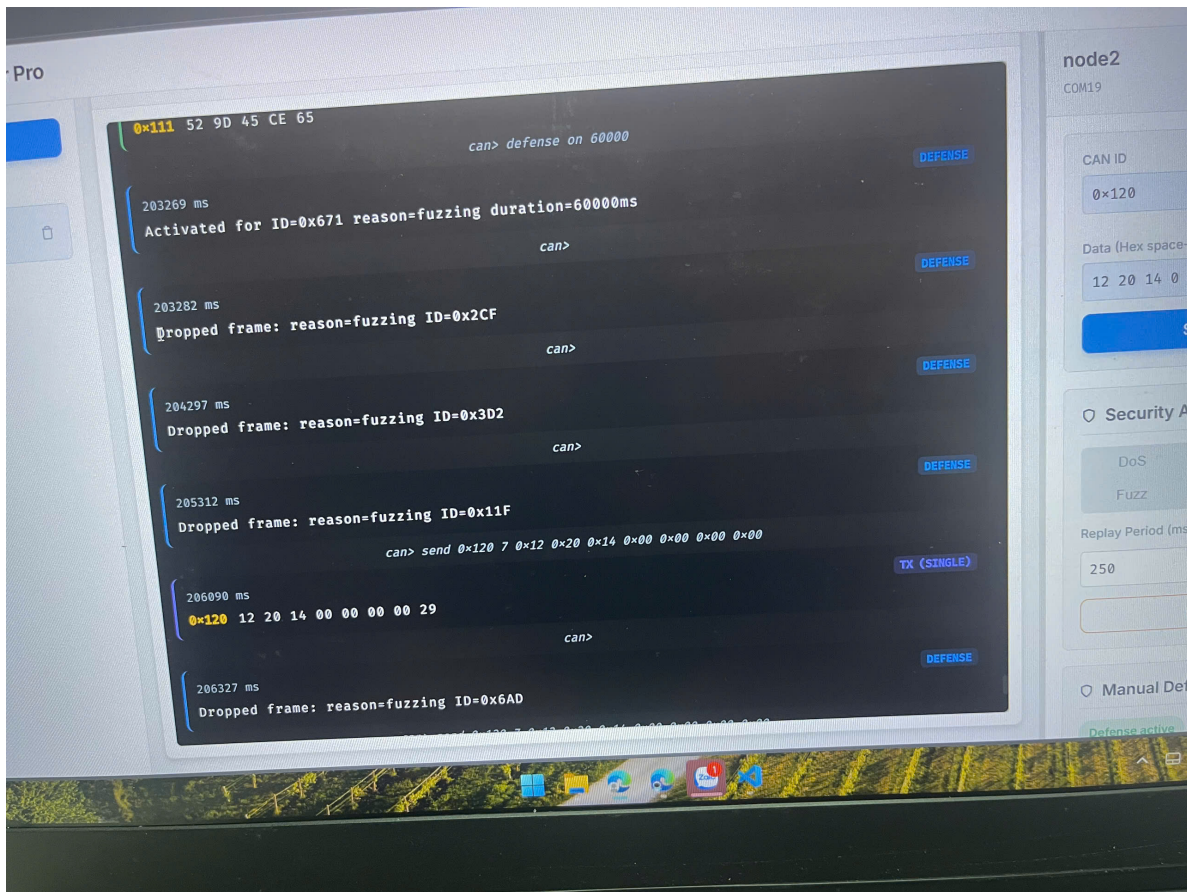


Figure 6.9: Giao diện sau khi bật defense để chặn lưu lượng Fuzzing

Bước cảnh báo: Ở Hình 6.8, nút nhận quan sát liên tiếp nhiều khung với CAN ID thay đổi nhanh như 0x176, 0x0E1, 0x1A5, 0x6A9, 0x671. Khi số lượng ID khác nhau và số lần chuyển đổi ID vượt ngưỡng, hệ thống sinh cảnh báo FUZZING suspected: unique_ids=6 id_switches=5 window=174ms. Điều này cho thấy lưu lượng đang bị tiêm nhiễu với mẫu phát không còn ổn định như giao tiếp bình thường.

Bước phòng thủ: Ở Hình 6.9, ngay sau khi người vận hành tiến hành gửi lệnh defense on 60000, hệ thống lập tức xác nhận kích hoạt thông qua thông báo Activated for ID=0x671 reason=fuzzing duration=60000ms. Sau đó, nhiều khung lại tiếp tục xuất hiện trên bus nhưng đã bị chặn ngay ở tầng ứng dụng với các dòng ghi log Dropped frame: reason=fuzzing ID=... Kết quả này chứng minh cơ chế phòng thủ đã cô lập thành công luồng fuzzing mà vẫn cho phép các bản tin hợp lệ tiếp tục được quan sát.

4. Tấn công Spoofing

Trong kịch bản Spoofing, nút tấn công phát khung với cùng CAN ID của một thông điệp hợp lệ nhưng thay đổi phần dữ liệu tải để giả mạo nội dung. Trước khi bắt đầu phần này, nên cho nút hợp lệ gửi vài lần cùng một ID để hệ thống phía nhận có cơ sở xây dựng mẫu dữ liệu

nên.

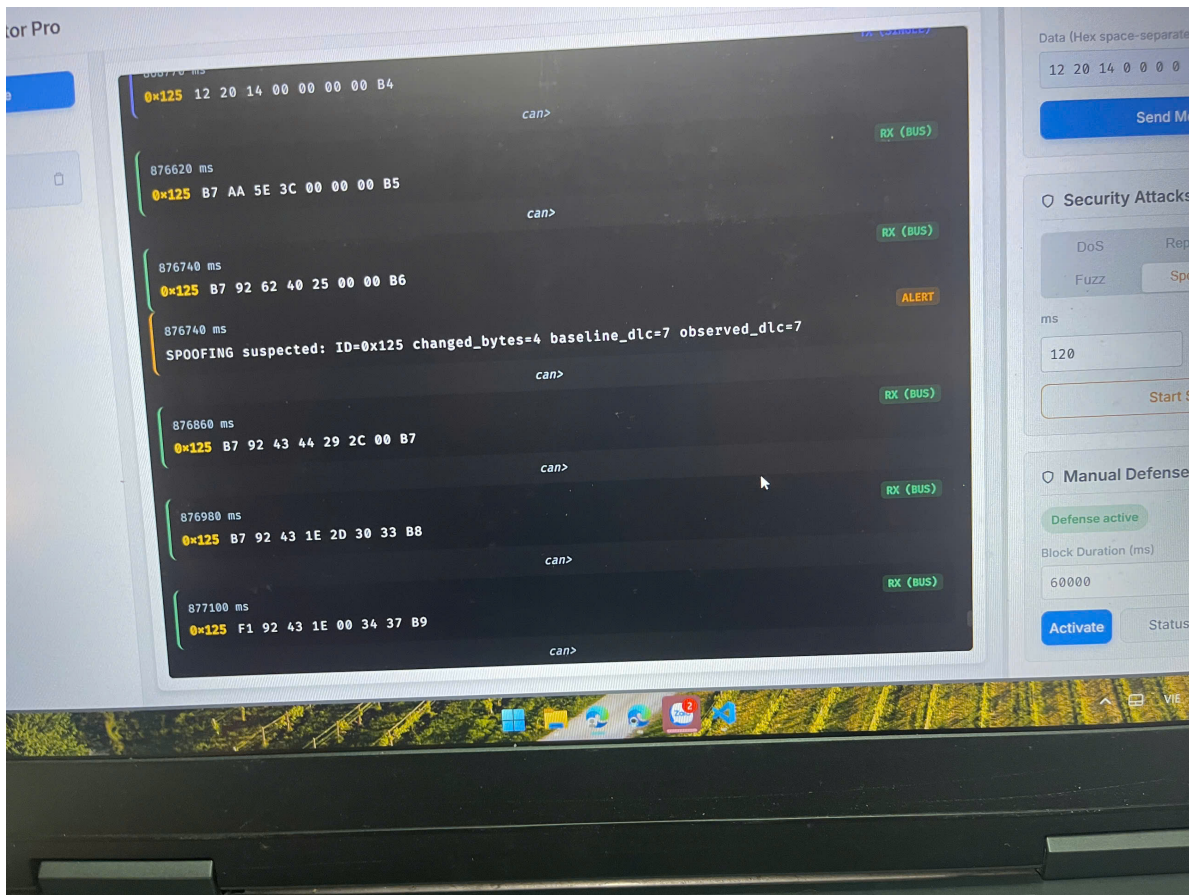


Figure 6.10: Giao diện ghi nhận cảnh báo ở giai đoạn phát hiện tấn công Spoofing

Khi spoofing diễn ra, phía nhận vẫn thấy cùng một CAN ID quen thuộc nhưng phần dữ liệu thay đổi bất thường so với mẫu hợp lệ trước đó. Đây là cơ sở để hệ thống sinh cảnh báo SPOOFING suspected và phân biệt khung giả mạo với khung hợp lệ có cùng ID.

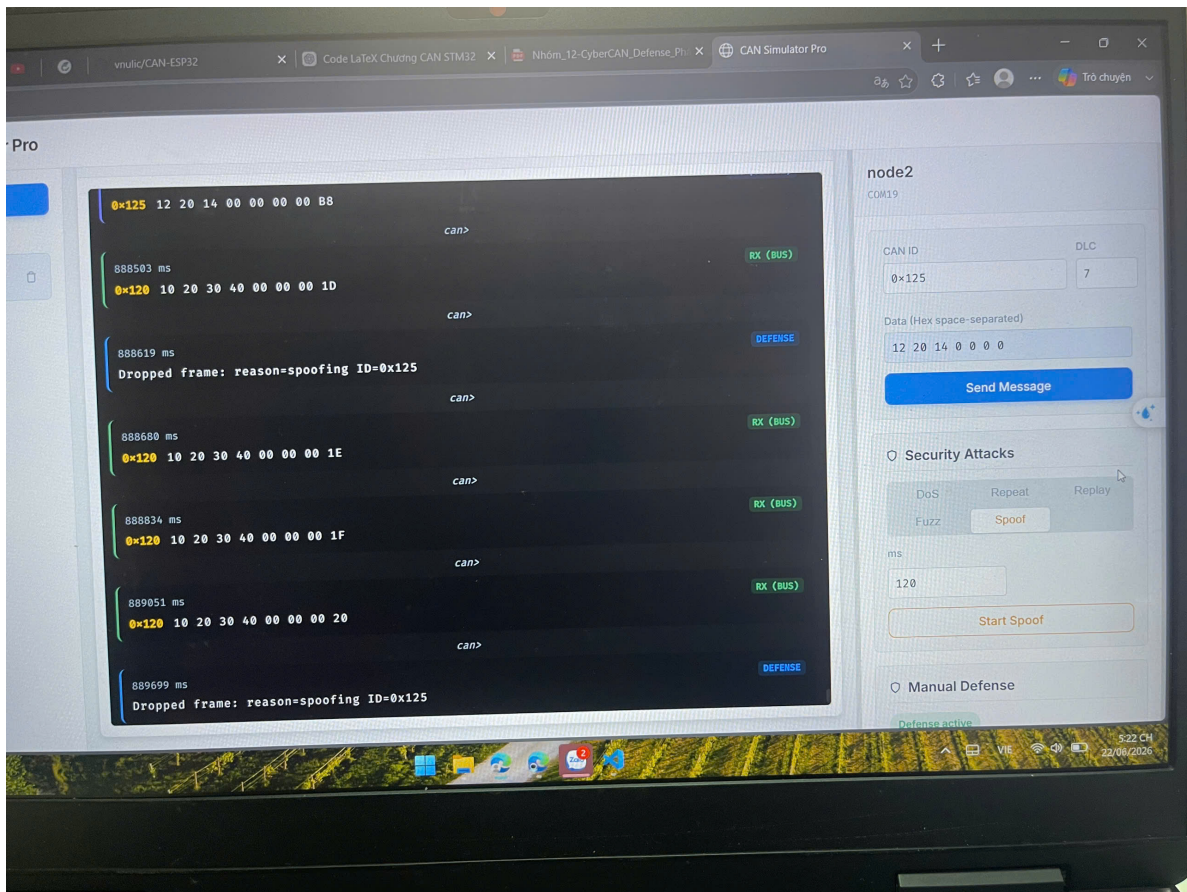


Figure 6.11: Giao diện sau khi bật defense để chặn khung spoofing

Bước cảnh báo: Ở Hình 6.10, nút nhận vẫn quan sát cùng CAN ID 0x125 nhưng nội dung payload đã thay đổi rõ rệt so với mẫu hợp lệ trước đó. Từ sự sai khác này, hệ thống sinh cảnh báo SPOOFING suspected: ID=0x125 changed_bytes=4 baseline_dlc=7 observed_dlc=7. Đây là dấu hiệu cho thấy một khung giả mạo đang cố chèn vào luồng giao tiếp bình thường.

Bước phòng thủ: Ở Hình 6.11, sau khi defense đã được bật, các khung giả mạo cùng ID 0x125 không còn được xử lý như dữ liệu hợp lệ mà bị ghi nhận bằng các dòng Dropped frame: reason=spoofing ID=0x125. Trong khi đó, các khung hợp lệ khác như 0x120 vẫn tiếp tục xuất hiện trên giao diện. Điều này cho thấy cơ chế defense đã chặn được luồng giả mạo theo đúng CAN ID nghi ngờ mà không làm gián đoạn hoàn toàn quá trình giám sát bus.

III. Video minh họa các kịch bản

Toàn bộ video minh họa cho các bước tấn công và phòng thủ của bốn kịch bản.

- Video tấn công và phòng thủ cho các kịch bản DoS, Replay, Fuzzing, Spoofing

Chapter 7

Tổng kết và hướng phát triển

7.1 Tổng kết

Bài tập lớn “CyberCAN Defense - Phát hiện và phòng thủ xâm nhập mạng nội bộ ô tô” tập trung nghiên cứu giao thức CAN, phân tích các điểm yếu bảo mật đặc trưng của bus này và xây dựng một mô hình giám sát, phát hiện, phòng thủ trên nền tảng ESP32 [1, 3, 4].

Trong quá trình thực hiện, nhóm đã đạt được các kết quả nổi bật sau:

- **Nghiên cứu lý thuyết nền tảng:** Nhóm đã hệ thống lại các kiến thức quan trọng về CAN như cấu trúc khung dữ liệu, cơ chế phân xử ưu tiên, truyền thông broadcast, phát hiện lỗi và yêu cầu lớp vật lý của bus [1].
- **Xây dựng mô hình hệ thống CAN thực tế:** Sử dụng các bo mạch vi điều khiển ESP32 và các module CAN của nhóm (dùng transceiver TJA1050 hoặc phần cứng tương đương), nhóm đã thiết lập mô hình bus CAN hai chiều giữa nhiều nút và máy tính giám sát [8, 9, 1]. Firmware được triển khai theo hướng gọn nhẹ, dễ kiểm thử và bám sát phần cứng thực tế của mô hình.
- **Thiết kế hệ thống truyền nhận cơ bản và có bảo vệ:**
 - Luồng truyền nhận CAN cơ bản được xây dựng để mô phỏng hoạt động bình thường giữa các nút trong mạng [3, 4].
 - Luồng có bảo vệ tích hợp bộ đếm thông điệp, phát hiện lặp khung, phát hiện ngập lưu lượng và cơ chế chặn theo CAN ID khi có cảnh báo [3, 4].
- **Mô phỏng tấn công và phòng thủ:**
 - Hệ thống hỗ trợ mô phỏng các kịch bản replay, spoofing theo chu kỳ và spam/DoS thông qua các lệnh điều khiển trên terminal [3, 4].
 - Cơ chế phòng thủ cho phép phát hiện bản tin bất thường và cô lập tạm thời các CAN ID bị nghi ngờ ở mức ứng dụng [3, 4].

- **Xây dựng giao diện trực quan bằng website:** Ứng dụng web dùng FastAPI, WebSocket và kết nối serial để theo dõi bản tin CAN, hiển thị cảnh báo, điều khiển tấn công và kích hoạt phòng thủ từ giao diện tập trung [11, 12].
- **Minh họa kết quả qua thực nghiệm:** Mô hình phần cứng và phần mềm của đề tài cho thấy khả năng trình diễn rõ ràng luồng truyền CAN cơ bản, kịch bản tấn công và phản ứng của cơ chế bảo vệ trên cùng một hệ thống.

Kết quả của dự án cho thấy cách tiếp cận phát hiện bất thường kết hợp chặn theo ID có thể nâng cao khả năng quan sát và giám sát động của các bản tin nghi ngờ trong mô hình CAN thử nghiệm [3, 4]. Đây là nền tảng phù hợp để tiếp tục phát triển các cơ chế bảo mật mạnh hơn cho mạng truyền thông nội bộ ô tô.

7.2 Hướng phát triển trong tương lai

Để mở rộng tiềm năng và ứng dụng thực tiễn của dự án, nhóm đề xuất một số hướng phát triển như sau:

- **Tăng cường bảo mật với mã hóa và xác thực:** Nghiên cứu các thuật toán mật mã nhẹ hoặc mã xác thực thông điệp phù hợp với giới hạn tài nguyên của vi điều khiển và kích thước khung CAN 2.0.
- **Nâng cấp cơ chế phát hiện bất thường:** Mở rộng IDS theo hướng kết hợp thống kê thời gian thực với mô hình học máy để nhận biết tốt hơn các mẫu tấn công chưa xuất hiện trong tập luật hiện tại.
- **Tối ưu cơ chế phòng thủ:** Bổ sung chính sách chặn theo thời gian thích nghi, phân biệt mức độ nghiêm trọng của cảnh báo và tránh chặn nhầm các bản tin hợp lệ.
- **Phát triển dashboard thời gian thực:** Hoàn thiện giao diện giám sát với biểu đồ lưu lượng theo CAN ID, lịch sử cảnh báo và nhật ký sự kiện phục vụ đánh giá sau thực nghiệm.
- **Mở rộng mô hình phần cứng:** Thử nghiệm với nhiều nút hơn, nhiều loại tải hơn hoặc gateway trung gian để mô phỏng gần hơn kiến trúc mạng trong ô tô thực.
- **Hướng đến hệ thống đa giao thức:** Bên cạnh CAN, có thể mở rộng nghiên cứu sang LIN, CAN FD, FlexRay hoặc Automotive Ethernet để hình thành cách tiếp cận bảo mật toàn diện hơn.

Với xu hướng phát triển mạnh của ô tô điện, xe kết nối và các hệ thống điều khiển ngày càng thông minh, bảo mật mạng truyền thông trong xe sẽ tiếp tục là yêu cầu quan trọng. Đề tài CyberCAN Defense là bước khởi đầu tốt để nhóm tiếp tục phát triển các mô hình bảo mật nhúng gắn với phần cứng và mã nguồn do chính nhóm làm chủ.

Tài liệu tham khảo

- [1] Robert Bosch GmbH, *CAN Specification Version 2.0*, 1991.
- [2] International Organization for Standardization, *ISO 11898-1: Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*, ISO, 2015.
- [3] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, *Experimental Security Analysis of a Modern Automobile*, IEEE Symposium on Security and Privacy, 2010.
- [4] C. Miller and C. Valasek, *Remote Exploitation of an Unaltered Passenger Vehicle*, Black Hat USA, 2015.
- [5] T. Hoppe, S. Kiltz, and J. Dittmann, *Security Threats to Automotive CAN Networks – Practical Examples and Selected Short-Term Countermeasures*, Computer Safety, Reliability, and Security, 2008.
- [6] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, and T. Kohno, *Comprehensive Experimental Analyses of Automotive Attack Surfaces*, USENIX Security Symposium, 2011.
- [7] AUTOSAR, *Specification of Secure Onboard Communication*, AUTOSAR Classic Platform.
- [8] Espressif Systems, *ESP-IDF Programming Guide: TWAI Driver*,
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/twai.html>.
- [9] NXP Semiconductors, *TJA1050 High-Speed CAN Transceiver Datasheet*.
- [10] Sandeep Mistry, *Arduino CAN Library*,
<https://github.com/sandeepmistry/arduino-CAN>.
- [11] Sebastian Ramirez, *FastAPI Documentation*, <https://fastapi.tiangolo.com/>.
- [12] PySerial Project, *PySerial Documentation*,
<https://pyserial.readthedocs.io/>.